ISTQB® Certified Tester

Foundation Level Specialist — **Performance Testing** (CT-PT, Syllabus 2018)

CHAPTER 3
Performance Testing in the Software Lifecycle

Complete Study Guide

Chapter fact	Value
Syllabus sections covered	3.1 Principal Performance Testing Activities · 3.2 Categories of Performance Risks for Different Architectures · 3.3 Performance Risks Across the Software Development Lifecycle · 3.4 Performance Testing Activities
Learning objectives	4 examinable (PTFL-3.1.1 – PTFL-3.4.1) — including the **first two K4 objectives** on the paper
Recommended teaching time	195 minutes — 2.2 marks per teaching hour, the **lowest ratio in the syllabus**
Questions in the exam	**7 of 40 questions · 7 of 40 points.** K1 = 0 · **K2 = 5** · K3 = 0 · **K4 = 2**. Every objective is guaranteed; **PTFL-3.2 gets three questions** and **PTFL-3.1 gets two**
Exam format	40 questions · 40 points · **90 minutes** (113 with the 25% non-native-language allowance) · pass mark 65% = **26 points**
Keywords	metric · risk · software development lifecycle · test log
Official sample-exam questions	**#11 – #17**. Questions #16 and #17 introduce the **regional health-records scenario** that recurs throughout Chapter 4

Built exclusively from the ISTQB® CT-PT 2018 syllabus, the official Sample Exam (questions and answers), and the ISTQB® exam structure tables.

Study this single file to master Chapter 3.

Contents

The page numbers above are filled in by Word: right-click the table of contents and choose "Update Field" → "Update entire table".

How to use this guide

Section 1 explains all four learning objectives in syllabus order — read it once, slowly. **Section 2** compresses the same material into recall-ready notes. **Section 3** holds the tables you should be able to reproduce from memory; **table 3.1 (the architecture risk grid)** and **table 3.2 (which activity owns this task)** carry five of the seven marks between them. **Section 4** is the question bank — attempt every question with the green answer box covered, then read the rationale even when you were right. **Section 5** converts that into exam tactics. **Section 6** is for daily spaced repetition. **Section 7** is what you read on the morning of the exam.

What makes Chapter 3 unusual: it is where the paper **stops being recall and starts being analysis**. The first two **K4** questions on the exam live here, and both are **scenario** items built on the regional health-records project that also supplies eight of Chapter 4's questions. The chapter is also the **worst points-per-hour investment in the syllabus — 7 marks for 195 teaching minutes** — so study it thoroughly but do not let it crowd out Chapter 4.

Three facts that decide five of the seven marks: first, **PTFL-3.2 is worth three marks** and two of its questions are **multiple-selection** items answered from one table — **connectivity risk means mobile and embedded**, and **memory-leak risk means virtualized, client-server, mobile and embedded**. Second, **PTFL-3.1 is worth two marks**, and both official questions name a **task** and ask which **activity** owns it — **risk identification belongs to test planning**, **ordering test cases into procedures belongs to test implementation**. Third, the third PTFL-3.2 mark was spent not on architectures at all but on **languages**: **C and C++ require you to monitor memory use; Python does not**.

The two K4 answer patterns. For **PTFL-3.3.1**, a timing question is won by the option that spans the **whole lifecycle** and says **repeatedly** — because risks **change in both impact and likelihood**. For **PTFL-3.4.1**, an approach question is won by the option that pairs an **early static review** with a **later dynamic test** — a single technique is almost always incomplete.

Suggested schedule: Day 1 — Section 1 for PTFL-3.1 and 3.2, then Section 3 tables 3.1 and 3.2. Day 2 — Section 1 for the two K4 objectives, plus Section 2. Day 3 — rebuild the architecture risk grid from blank paper, then Section 4 Part A (all seven official questions). Day 4 — Section 4 Part B, plus Section 5. Day 5 onwards — Section 6 flashcards daily, Section 7 the night before.

Section 1 — Syllabus Explanation

Chapter 3 carries **4 learning objectives** across four sub-chapters (3.1 Principal Performance Testing Activities · 3.2 Categories of Performance Risks for Different Architectures · 3.3 Performance Risks Across the Software Development Lifecycle · 3.4 Performance Testing Activities). The official ISTQB Exam Structure Table allocates **7 of the 40 exam questions** to Chapter 3 — **7 of the 40 points**. Recommended teaching time is **195 minutes**.

This is where the paper stops being recall and starts being analysis. Chapter 3 contains the **first two K4 questions on the exam** — PTFL-3.3 and PTFL-3.4 — and they are scenario questions built on the same health-records project that runs through most of Chapter 4. Five of the seven marks are still K2, and **PTFL-3.2 alone is worth three of them**, making it the second-largest single-objective allocation on the whole paper.

LO ID	K	Objective (syllabus statement)	Qs	Pts
PTFL-3.1.1	K2	Understand the principal performance testing activities.	**2**	2
PTFL-3.2.1	K2	Explain typical categories of performance risks for different architectures.	**3**	3
PTFL-3.3.1	**K4**	Analyze performance risks for a given product across the software development lifecycle.	**1**	1
PTFL-3.4.1	**K4**	Analyze a given project to determine the appropriate performance testing activities for each phase of the software development lifecycle.	**1**	1
Chapter 3 total	—	**K1 = 0 · K2 = 5 · K3 = 0 · K4 = 2**	**7**	**7**

Exam allocation you must know

Chapter 3 = **7 questions / 7 points** of 40 questions and 40 points. In the official Sample Exam these are questions **#11 to #17**. **PTFL-3.2 receives three questions — more than any objective outside Chapter 4.** The two K4 questions (#16, #17) are the first scenario-based items on the paper and both use the **regional patient health-records** scenario.

The one thing that decides this chapter

Three of the seven marks come from one table: which performance risks belong to which architecture. The official exam asks it three times in a row, twice in a "*consider the following technical environments… which of these is most likely to have a risk due to X*" multiple-selection format. **Nine architectures, each with two or three named risks.** Build that table, rebuild it from blank paper, and you have taken 43% of the chapter.

PTFL-3.1.1 (K2) — Principal Performance Testing Activities

Original syllabus statement

Understand the principal performance testing activities.

The framing sentence. "*Performance testing is **iterative in nature**. **Each test provides valuable insights** into application and system performance. The information gathered from one test is used to **correct or optimize application and system parameters**. The next test iteration will then **show the results of modifications**, and so on **until test objectives are reached***."

The alignment sentence. "*Performance testing activities **align with the ISTQB test process***." That means the seven activities below are the standard Foundation-Level test process activities, seen through a performance lens. The exam asks **which activity owns a named task** — so the discriminator is always the task, never the definition of the activity.

The seven activities — and what belongs to each

Activity	What the syllabus places here
Test planning	**Particularly important for performance testing** because of the need for the **allocation of test environments, test data, tools and human resources**. This is the activity in which **the scope of performance testing is established**. **During test planning, risk identification and risk analysis activities are completed** and relevant information is updated in test planning documentation (e.g., **test plan, level test plan**). **Just as test planning is revisited and modified as needed, so are risks, risk levels and risk status modified to reflect changes in risk conditions.**
Test monitoring and control	**Control measures are defined to provide action plans should issues be encountered** which might impact performance efficiency: **increasing the load generation capacity** if the infrastructure does not generate the desired loads · **changed, new or replaced hardware** · **changes to network components** · **changes to software implementation**. **The performance test objectives are evaluated to check for exit criteria achievement.**
Test analysis	**Effective performance tests are based on an analysis of performance requirements, test objectives, Service Level Agreements (SLA), IT architecture, process models and other items that comprise the test basis.** May be supported by **modeling and analysis of system resource requirements and/or behavior using spreadsheets or capacity planning tools**. **Specific test conditions are identified** such as **load levels, timing conditions, and transactions to be tested**. **The required type(s) of performance test (e.g., load, stress, scalability) are then decided.**
Test design	**Performance test cases are designed.** These are **generally created in modular form** so that they may be used as the **building blocks of larger, more complex performance tests** (see §4.2).
Test implementation	**Performance test cases are ordered into performance test procedures.** These procedures should **reflect the steps normally taken by the user and other functional activities** to be covered. A test implementation activity is **establishing and/or resetting the test environment before each test execution**. Since performance testing is **typically data-driven**, a process is needed to **establish test data that is representative of actual production data in volume and type** so that **production use can be simulated**.
Test execution	**Test execution occurs when the performance test is conducted, often by using performance test tools. Test results are evaluated to determine if the system's performance meets the requirements and other stated objectives. Any defects are reported.**
Test completion	**Performance test results are provided to the stakeholders** (e.g., **architects, managers, product owners**) in a **test summary report**. The results are **expressed through metrics which are often aggregated to simplify the meaning** of the test results. **Visual means of reporting such as dashboards** are often used. At the close of a defined period, a **point of test closure** may be reached where **designed tests, test tool assets (test cases and test procedures), test data and other testware are archived or passed on to other testers for later use during system maintenance activities**.

The two official questions, and the exact tasks they turn on

#11 — "When should risk identification and analysis occur?" → Test planning. The official rationale: **risk identification and analysis is done at the test planning stage. This information is then used for the analysis and design of the tests.** Analysis and design, implementation and execution, and closure are all **too late in the process**. **#12 — "When should the test cases be ordered into performance test procedures?" → Test implementation and execution.** The rationale: **test planning and test analysis and design are when the test cases are planned and designed**; **test closure is too late**. Note the option wording pairs activities — *test implementation and execution* is offered as one option.

The ongoing-activity sentence. "*Performance testing is often considered to be an **ongoing activity** in that it is **performed at multiple times and at all test levels (component, integration, system, system integration and acceptance testing)***." This is the bridge back to **PTFL-1.3.1** and forward to **PTFL-3.4.1**.

Common misunderstanding. Reading "test analysis" as the place where risks are identified. Risks are identified and analysed in **test planning**; test **analysis** is where the **test basis** is analysed and **test conditions** — load levels, timing conditions, transactions — are identified, and where the **type of performance test is decided**.

Related concepts: iterative testing, ISTQB test process, test planning, risk identification, risk analysis, test plan, level test plan, test monitoring and control, control measures, exit criteria, test analysis, test basis, SLA, capacity planning tools, test conditions, test design, modular test cases, test implementation, test procedures, data-driven testing, test execution, test completion, test summary report, dashboards, test closure, testware archiving.

PTFL-3.2.1 (K2) — Categories of Performance Risks for Different Architectures

Original syllabus statement

Explain typical categories of performance risks for different architectures.

The framing. "*Application or system performance* ***varies considerably based on the architecture, application and host environment***. *While* ***it is not possible to provide a complete list of performance risks for all systems***, *the list below includes* ***some typical types of risks associated with particular architectures***." Nine architectures are named. **Three of your seven chapter marks are decided here.**

The nine architectures and their named risks

Architecture	Definition	Performance risks named by the syllabus
Single computer systems	Systems or applications that run **entirely on one non-virtualized computer**.	**Excessive resource consumption** including **memory leaks, background activities such as security software, slow storage subsystems** (low-speed external devices or **disk fragmentation**), and **operating system mismanagement**. **Inefficient implementation of algorithms** which **do not make use of available resources (e.g., main memory)** and so **execute slower than required**.
Multi-tier systems	**Systems of systems that run on multiple servers**, each performing a specific set of tasks — **database server, application server, presentation server**. Each server is itself a computer and **subject to the single-computer risks**.	**Poor or non-scalable database design · network bottlenecks · inadequate bandwidth or capacity on any single server.**
Distributed systems	Similar to multi-tier, but **the various servers may change dynamically** — e.g., an e-commerce system accessing **different inventory databases depending on the geographic location** of the person ordering. **Inherits all multi-tier risks.**	**Critical workflows or dataflows to, from, or through unreliable or unpredictable remote servers**, especially when such servers suffer **periodic connection problems or intermittent periods of intense load**.
Virtualized systems	Systems where the **physical hardware hosts multiple virtual computers**. These VMs may host single-computer systems and applications as well as servers that are part of a multi-tier or distributed architecture.	**Excessive load on the hardware across all the virtual machines**; **improper configuration of the host virtual machine resulting in inadequate resources**.
Dynamic / cloud-based systems	Systems that **offer the ability to scale on demand**, increasing capacity as load increases. **Typically distributed and virtualized multi-tier systems**, with **self-scaling features designed specifically to**	**Failures to properly configure these features during initial setup or subsequent updates.**

Architecture	Definition	Performance risks named by the syllabus
	mitigate some of the performance risks of those architectures.	
Client-server systems	Systems running on a client that **communicate via a user interface with a single server, multi-tier server, or distributed server. Since there is code running on the client, the single-computer risks apply to that code**, while the server-side issues apply as well.	**Connection speed and reliability issues · network congestion at the client connection point (e.g., public Wi-Fi) · potential problems due to firewalls, packet inspection and server load balancing.**
Mobile applications	Applications running on a **smartphone, tablet, or other mobile device**. Subject to the risks of **client-server and browser-based (web apps)** applications.	**Limited and variable resources and connectivity** — affected by **location, battery life, charge state, available memory on the device and temperature**. For apps using **device sensors or radios such as accelerometers or Bluetooth**, **slow dataflows from those sources**. **Heavy interactions with other local mobile apps and remote web services**, any of which can become a **performance efficiency bottleneck**.
Embedded real-time systems	Systems that **work within or even control everyday things** — cars (entertainment systems, **intelligent braking systems**), **elevators, traffic signals, HVAC systems**.	**Many of the risks of mobile devices**, including **(increasingly) connectivity-related issues** since these devices are connected to the Internet. **However** — the diminished performance of a mobile video game is usually **not a safety hazard**, while such slowdowns in a **vehicle braking system could prove catastrophic**.
Mainframe applications	**In many cases decades-old applications** supporting **mission-critical business functions in a data center, sometimes via batch processing. Most are quite predictable and fast when used as originally designed.**	**Many are now accessible via APIs, web services, or through their database**, which can result in **unexpected loads that affect throughput of established applications**.

The two multiple-selection questions, decoded

#13 — which environments are most likely to have a risk due to connectivity issues? Answer: mobile and embedded. The official rationale: **mobile and embedded environments are particularly prone to connectivity issues, particularly when embedded software is used in IoT devices**; the other environments **are not prone to connection dropping**. **#14 — which are most likely to have a risk due to memory leaks? Answer: virtualized, client/server and browser-based, mobile, embedded.** The reasoning is given per environment: **virtualized uses shared resources across different applications, including memory**; **dynamic/cloud-based is designed to dynamically scale, so while memory leaks may occur they are less of a risk because the environment will scale to compensate**; **client/server and browser-based are prone to memory leaks, particularly since a lot of this code is written in C and C++**; **mobile is prone to leakage and particularly at risk due to the limited available memory**; **embedded likewise, and these also tend to be written in languages that have less protection against leaks**; **mainframes tend to run older code, often written in the higher level languages that provide their own memory management**.

Architectures combine. "*Note that **any particular application or system may incorporate two or more of the architectures listed above**, which means that **all relevant risks will apply**. In fact, given the **Internet of Things and the explosion of mobile applications** — two areas where **extreme levels of interaction and connection is the rule — it is possible that all architectures are present in some form in an application, and thus all risks can apply**.*"

Risks that are not architectural — the third official question

"*While architecture is clearly an important technical decision with a profound impact on performance risks, **other technical decisions also influence and create risks**.*" The syllabus names three: **memory leaks are more common with languages that allow direct heap memory management, such as C and C++ · performance issues are different for relational versus non-relational**

databases · and such decisions **extend all the way down to the design of individual functions or methods (e.g., the choice of a recursive as opposed to an iterative algorithm)**. **Official question #15 is built entirely on the first of these:** if the software is written in **C++** rather than **Python**, you must monitor **memory use**, because **C and C++ require the programmer to do their own memory management, which sometimes results in memory leaks (where memory is allocated, but not de-allocated). Network latency, connectivity and batch processing would have the same issues regardless of the language used.**

A closing caution from the syllabus. "*As a tester, the ability to **know about or even influence such decisions will vary, depending on the roles and responsibilities of testers within the organization and software development lifecycle***."

Related concepts: single computer system, memory leak, background activity, disk fragmentation, operating system mismanagement, algorithm efficiency, multi-tier system, database server, application server, presentation server, network bottleneck, distributed system, remote server reliability, virtualized system, host virtual machine configuration, dynamic/cloud-based system, self-scaling, client-server system, public Wi-Fi congestion, firewall, packet inspection, load balancing, mobile application, battery life, charge state, temperature, accelerometer, Bluetooth, embedded real-time system, intelligent braking system, HVAC, safety hazard, mainframe application, batch processing, API exposure, heap memory management, C and C++, relational versus non-relational databases, recursive versus iterative algorithms.

PTFL-3.3.1 (K4) — Performance Risks Across the Software Development Lifecycle

Original syllabus statement

Analyze performance risks for a given product across the software development lifecycle.

This is the first K4 objective on the paper. K4 means **analyze** — the candidate can **separate information related to a procedure or technique into its constituent parts for better understanding and can distinguish between facts and inferences** In practice the question gives you a project description and asks **when** risk analysis should happen.

The four-step performance risk process

#	Step (syllabus wording)
1	**Identify risks to product quality**, focusing on characteristics such as **time behavior, resource utilization, and capacity**.
2	**Assess the identified risks**, ensuring that the **relevant architecture categories (see Section 3.2) are addressed**. **Evaluate the overall level of risk for each identified risk in terms of likelihood and impact using clearly defined criteria.**
3	**Take appropriate risk mitigation actions for each risk item** based on the **nature of the risk item and the level of risk**.
4	**Manage risks on an ongoing basis** to ensure that the risks are **adequately mitigated prior to release**.

Who takes part, and what each side must appreciate. The participants "*should include **both business and technical stakeholders***", and the syllabus is precise about what each group brings.

Stakeholder group	Who must be included	What they must appreciate
Business stakeholders	Those with **a particular awareness of how performance problems in production will actually affect customers, users, the business, and other downstream stakeholders**.	That **intended usage, business-, societal-, or safety-criticality, potential financial and/or reputational damage, civil or criminal legal liability** and similar factors **affect risk from a business perspective, creating risks and influencing the impact of failures**.
Technical stakeholders	Those with **a deep understanding of the performance implications of relevant requirements, architecture, design, and implementation decisions**.	That **architecture, design, and implementation decisions affect performance risks from a technical perspective, creating risks and influencing the likelihood of defects**.

Impact versus likelihood — the split the exam relies on

Business stakeholders influence the IMPACT of failures. Technical stakeholders influence the LIKELIHOOD of defects. That is the syllabus's own division, and it is the cleanest way to remember why both groups are mandatory. Step 2 evaluates risk **in terms of likelihood and impact using clearly defined criteria** — one side of that equation comes from each group.

Formality and timing. "*The specific risk analysis process chosen should have* ***the appropriate level of formality and rigor****. For performance-related risks, it is* ***especially important that the risk analysis process be started early and is repeated regularly****. In other words,* ***the tester should avoid relying entirely on performance testing conducted towards the end of the system test level and system integration test level****.*"

Why late is expensive. "*Many projects, especially* ***larger and more complex systems of systems projects****, have met with* ***unfortunate surprises due to the late discovery of performance defects which resulted from requirements, design, architecture, and implementation decisions made early in the project****. The emphasis should therefore be on an* ***iterative approach to performance risk identification, assessment, mitigation, and management throughout the software development lifecycle****.*"

The worked example the syllabus gives

"*If* ***large volumes of data will be handled via a relational database****, the* ***slow performance of many-to-many joins due to poor database design*** *may* ***only reveal itself during dynamic testing with large-scale test datasets, such as those used during system test. However, a careful technical review that includes experienced database engineers can predict the problems prior to database implementation.*** *After such a review,* ***in an iterative approach, risks are identified and assessed again****.*" This is **PTFL-1.3.1's static testing argument** restated as a risk-management argument — and it is the reasoning behind the answer to official question #17.

Mitigation spans the whole process. "*Risk mitigation and management* ***must span and influence the entire software development process, not just dynamic testing****.*" The syllabus example: when **critical performance-related decisions such as the expected number of transactions or simultaneous users cannot be specified early**, it is important that **design and architecture decisions allow for highly variable scalability (e.g., on-demand cloud-based computing resources). This enables early risk mitigation decisions to be made.**

Residual risk must be quantified. "*As with any type of quality risk, performance-related risks* ***can never be avoided completely*** *— i.e.,* ***some risk of performance-related production failure will always exist****. Therefore, the risk management process* ***must include providing a realistic and specific evaluation of the residual level of risk*** *to the business and technical stakeholders involved.*" The syllabus then gives its own bad example and good example:

Unhelpful residual-risk statement	Helpful residual-risk statement
"Yes, it's still possible for customers to experience long delays during check out" — not helpful, as it gives no idea of what amount of risk mitigation has occurred or of the level of risk that remains	**Providing clear insight into the percentage of customers likely to experience delays equal to or exceeding certain thresholds** — this **helps people understand the status**

Exam focus. One **K4** question. The official version presents the **regional patient health-records** scenario — millions of records, doctors in offices, hospitals and urgent care facilities, **information presented within three seconds, particularly for patients with critical allergies and preconditions** — and asks **when is the best time in the project to analyze and assess the performance risks?** The answer is **repeatedly throughout the requirements, development and performance testing**. The rationale: **since performance is critical, the risks must be analyzed as early as possible and then repeatedly as the system is assembled, since risks may change both in impact and likelihood**. Each distractor **leaves out a phase**: one **ignores looking at the risks during development**; one **leaves out the requirements and development**; one **leaves out the requirements and development options for analysis**.

The K4 answer pattern for PTFL-3.3.1

When a performance-risk timing question offers one option that spans the whole lifecycle and three that name only some phases, the spanning option wins. The syllabus sentence behind it is: **started early and repeated regularly**, with an **iterative approach to identification, assessment, mitigation and management throughout the software development lifecycle**. Also note the reason risks must be **re-assessed**, not merely re-checked: **risks may change both in impact and likelihood**.

Related concepts: quality risk analysis, risk identification, risk assessment, likelihood, impact, risk mitigation, ongoing risk management, residual risk, business stakeholders, technical stakeholders, safety-criticality, reputational damage, legal liability, formality and rigor, systems of systems, many-to-many joins, technical review, on-demand cloud-based computing, thresholds.

PTFL-3.4.1 (K4) — Performance Testing Activities Across Lifecycle Models

Original syllabus statement

Analyze a given project to determine the appropriate performance testing activities for each phase of the software development lifecycle.

The framing. "*Performance testing activities will be* ***organized and performed differently, depending on the type of software development lifecycle in use***." Three lifecycle families are covered: **sequential**, **iterative and incremental**, and **COTS / supplier-acquirer models**.

Sequential development models

"*The* ***ideal practice*** *of performance testing in sequential development models is to* ***include performance criteria as a part of the acceptance criteria which are defined at the outset of a project***. *Reinforcing the lifecycle view of testing,* ***performance testing activities should be conducted throughout the software development lifecycle***. *As the project progresses,* ***each successive performance test activity should be based on items defined in the prior activities***." **Nine phases**, each with its own verb — and the verb is what the exam tests.

Phase	Performance testing activity (syllabus wording)
Concept	**Verify that system performance goals are defined as acceptance criteria for the project.**
Requirements	**Verify that performance requirements are defined and represent stakeholder needs correctly.**
Analysis and Design	**Verify that the system design reflects the performance requirements.**
Coding / Implementation	**Verify that the code is efficient and reflects the requirements and design in terms of performance.**
Component Testing	**Conduct component level performance testing.**
Component Integration Testing	**Conduct performance testing at the component integration level.**
System Testing	**Conduct performance testing at the system level**, which includes **hardware, software, procedures and data that are representative of the production environment**. **System interfaces may be simulated provided that they give a true representation of performance.**
System Integration Testing	**Conduct performance testing with the entire system which is representative of the production environment.**
Acceptance Testing	**Validate that system performance meets the originally stated user needs and acceptance criteria.**

Verify versus validate — and where simulation is allowed

The first four phases all say **verify**; the last says **validate**. **Verification checks the artefact against its predecessor; validation checks the system against the originally stated user needs and acceptance criteria.** Note also the single place the syllabus permits simulation: at **system testing**, **system interfaces may be simulated provided that they give a true representation of performance** — but at **system integration testing** the requirement is **the entire system which is representative of the**

production environment.

Iterative and incremental development models

"*In these development models, such as* ***Agile****, performance testing is also seen as an* ***iterative and incremental activity****. Performance testing* ***can occur as part of the first iteration, or as an iteration dedicated entirely to performance testing****. However, with these lifecycle models,* ***the execution of performance testing may be performed by a separate team tasked with performance testing***."

Continuous Integration. "*CI is commonly performed in iterative and incremental software development lifecycles, which* ***facilitates a highly automated execution of tests****.* ***The most common objective of testing in CI is to perform regression testing and ensure each build is stable.*** *Performance testing* ***can be part of the automated tests performed in CI if the tests are designed in such a way as to be executed at a build level****. However,* ***unlike functional automated tests, there are additional concerns***" — four of them:

CI concern	What the syllabus says
The setup of the performance test environment	**Often requires a test environment that is available on demand, such as a cloud-based performance test environment.**
Determining which performance tests to automate in CI	**Due to the short timeframe available for CI tests**, CI performance tests **may be a subset of more extensive performance tests conducted by a specialist team at other times during an iteration**.
Creating the performance tests for CI	**The main objective of performance tests as part of CI is to ensure a change does not negatively impact performance. Depending on the changes made for any given build, new performance tests may be required.**
Executing performance tests on portions of an application or system	**Often requires the tools and test environments to be capable of rapid performance testing, including the ability to select subsets of applicable tests.**

The seven iterative lifecycle activities. "*Performance testing in the iterative and incremental software development lifecycles can also have* ***its own lifecycle activities***."

Activity	Performance testing content
Release Planning	Performance testing is considered **from the perspective of all iterations in a release, from the first iteration to the final iteration**. **Performance risks are identified and assessed, and mitigation measures planned.** Often includes **planning of any final performance testing before the release of the application**.
Iteration Planning	Performance testing **may be performed within the iteration and as each iteration is completed**. **Performance risks are assessed in more detail for each user story.**
User Story Creation	**User stories often form the basis of performance requirements in Agile methodologies**, with the **specific performance criteria described in the associated acceptance criteria**. These are referred to as **"non-functional" user stories**.
Design of performance tests	**Performance requirements and criteria which are described in particular user stories are used for the design of tests** (see §4.2).
Coding / Implementation	**Performance testing may be performed at a component level** — for example, **the tuning of algorithms for optimum performance efficiency**.
Testing / Evaluation	**While testing is typically performed in close proximity to development activities, performance testing may be performed as a separate activity**, depending on the **scope and objectives of performance testing during the iteration**. If the goal is to test **the iteration as a completed set of user stories**, a **wider scope** is needed than for a **single user story**. **This may be scheduled in a dedicated iteration for performance testing.**
Delivery	**Since delivery will introduce the application to the production environment, performance will need to**

Activity	Performance testing content
	be monitored to determine if the application achieves the desired levels of performance in actual usage.

COTS and other supplier / acquirer models

"*Many organizations **do not develop applications and systems themselves**, but instead are in the position of **acquiring software from vendor sources or from open-source projects**. In such **supplier/acquirer models**, performance is an important consideration that **requires testing from both the supplier (vendor/developer) and acquirer (customer) perspectives**.*"

Situation	Where responsibility sits
Regardless of the source of the application	**It is often the responsibility of the customer to validate that the performance meets their requirements.**
Customized vendor-developed software	**Performance requirements and associated acceptance criteria should be specified as part of the contract between the vendor and customer.**
COTS applications	**The customer has sole responsibility to test the performance of the product in a realistic test environment prior to deployment.**

The official K4 question — and why the answer is a combination

Question #17 uses the same **health-records** scenario and asks: **what is the best way to address the performance testing for the response time?** The answer: **conduct a technical review of the database implementation and conduct a performance test from the UI with the full database loaded.** The official rationale: **the best way to do the testing is to first do a technical analysis because this can be done at design time. Once implemented, full testing will be needed to verify that the solution does indeed deliver the performance required.** The distractors fail as follows: testing from the UI with the full data set **leaves out the static testing**; testing via the web services at API level **would bypass any issues that might occur at the UI end and doesn't consider the database loading**; and the network assessment option is wrong because **the network is less likely an issue regarding the data handling for such a large amount of data**.

The K4 answer pattern for PTFL-3.4.1

When asked how to address a performance concern across a lifecycle, prefer the option that pairs an early STATIC activity with a later DYNAMIC one. Chapter 1 told you **static testing matters more for performance than for functional testing because critical defects are introduced in architecture and design**; Chapter 3 told you risk analysis must be **started early and repeated regularly**; and the official rationale rewards exactly that combination. A single-technique answer — UI only, API only, network only — is almost always incomplete.

Related concepts: sequential development model, acceptance criteria at project outset, concept phase, requirements phase, analysis and design, coding/implementation, component testing, component integration testing, system testing, simulated interfaces, system integration testing, acceptance testing, verification versus validation, iterative and incremental models, Agile, dedicated performance iteration, separate performance team, Continuous Integration, regression testing, build stability, on-demand cloud test environment, test subsets, release planning, iteration planning, non-functional user stories, algorithm tuning, delivery monitoring, COTS, supplier/acquirer model, contractual acceptance criteria.

Section 2 — Key Concepts

Concise, exam-ready notes drawn from the syllabus text. Every item is traceable to Chapter 3 of the CT-PT syllabus (pages 23–31).

2.1 The seven principal activities

- **Performance testing is iterative in nature** — each test's insights **correct or optimize application and system parameters**, and the next iteration **shows the results of modifications, until test objectives are reached**.
- **Performance testing activities align with the ISTQB test process.**
- **Test planning** — **particularly important** because of the **allocation of test environments, test data, tools and human resources**; **the scope of performance testing is established** here; **risk identification and risk analysis activities are completed** here; **risks, risk levels and risk status are modified** as conditions change.
- **Test monitoring and control** — **control measures define action plans** for **load generation capacity shortfalls · changed, new or replaced hardware · changes to network components · changes to software implementation**; **performance test objectives are evaluated to check for exit criteria achievement**.
- **Test analysis** — based on **performance requirements, test objectives, SLAs, IT architecture, process models and other test basis items**; may use **spreadsheets or capacity planning tools**; identifies **load levels, timing conditions and transactions to be tested**; **decides the required type(s) of performance test**.
- **Test design** — performance test cases are **generally created in modular form** as **building blocks of larger, more complex performance tests**.
- **Test implementation** — test cases are **ordered into performance test procedures** reflecting **the steps normally taken by the user**; **establishing and/or resetting the test environment before each execution**; **test data representative of actual production data in volume and type**.
- **Test execution** — the test is **conducted, often using performance test tools**; results **evaluated against requirements and objectives**; **defects reported**.
- **Test completion** — results provided to stakeholders (**architects, managers, product owners**) in a **test summary report**; metrics **often aggregated**; **dashboards** used; at **test closure**, **test cases, test procedures, test data and other testware are archived or passed on** for **system maintenance**.
- Performance testing is an **ongoing activity performed at multiple times and at all test levels**.

2.2 Nine architectures, nine risk profiles

- **Single computer** — **excessive resource consumption** (memory leaks, **background activities such as security software**, slow storage subsystems, **disk fragmentation**, **operating system mismanagement**) and **inefficient algorithms that do not use available resources**.
- **Multi-tier** — **poor or non-scalable database design · network bottlenecks · inadequate bandwidth or capacity on any single server** (plus all single-computer risks per server).
- **Distributed** — multi-tier risks plus **unreliable or unpredictable remote servers** with **periodic connection problems or intermittent intense load**; servers **may change dynamically**.
- **Virtualized** — **excessive load on the hardware across all the virtual machines**; **improper host VM configuration resulting in inadequate resources**.
- **Dynamic / cloud-based** — **self-scaling mitigates some risks**, but **failures to properly configure these features during initial setup or subsequent updates** remain.
- **Client-server** — client-side code carries **single-computer risks**; plus **connection speed and reliability**, **network congestion at the client connection point (public Wi-Fi)**, **firewalls, packet inspection and server load balancing**.
- **Mobile** — client-server and web-app risks plus **limited and variable resources and connectivity** (**location, battery life, charge state, available memory, temperature**), **slow dataflows from sensors and radios (accelerometers, Bluetooth)**, and **heavy interactions with other local apps and remote web services**.
- **Embedded real-time** — many mobile risks including **connectivity**, but **slowdowns can be catastrophic** (**vehicle braking**) rather than merely annoying.

- **Mainframe — predictable and fast as originally designed**, but now **accessible via APIs, web services or the database**, causing **unexpected loads that affect throughput of established applications**.
- **Any application may incorporate two or more architectures**, so **all relevant risks apply**; with **IoT and mobile**, **all architectures may be present and all risks can apply**.
- **Non-architectural technical decisions also create risk: C and C++ allow direct heap memory management → memory leaks · relational versus non-relational databases · recursive versus iterative algorithms**.

2.3 Performance risk across the lifecycle

- **Four steps: identify** risks focusing on **time behavior, resource utilization and capacity → assess**, ensuring **relevant architecture categories are addressed**, evaluating **likelihood and impact using clearly defined criteria → mitigate** based on **the nature of the risk item and the level of risk → manage on an ongoing basis** to ensure risks are **adequately mitigated prior to release**.
- **Both business and technical stakeholders** must participate. **Business stakeholders influence impact** (intended usage, business/societal/safety criticality, financial and reputational damage, civil or criminal legal liability). **Technical stakeholders influence likelihood** (requirements, architecture, design and implementation decisions).
- The process must have **the appropriate level of formality and rigor**, and must be **started early and repeated regularly**.
- **Do not rely entirely on performance testing conducted towards the end of system test and system integration test.**
- **Late discovery of performance defects** — from **requirements, design, architecture and implementation decisions made early** — has caused **unfortunate surprises**, especially in **larger and more complex systems of systems projects**.
- **Example: many-to-many joins** on a poorly designed relational database may only appear **during dynamic testing with large-scale test datasets at system test**, but **a careful technical review including experienced database engineers can predict the problems prior to database implementation**.
- **Mitigation must span the entire software development process, not just dynamic testing** — e.g., when transaction volumes or simultaneous-user counts **cannot be specified early**, **design and architecture decisions should allow for highly variable scalability (on-demand cloud-based computing resources)**.
- **Residual risk always exists** and must be given a **realistic and specific evaluation**. "**Yes, it's still possible for customers to experience long delays**" is unhelpful; **the percentage of customers likely to experience delays equal to or exceeding certain thresholds** is helpful.

2.4 Lifecycle models

- **Sequential — ideal practice: include performance criteria as part of the acceptance criteria defined at the outset of the project**; **each successive performance test activity is based on items defined in the prior activities**.
- **Nine sequential phases: Concept** (verify performance goals are defined as acceptance criteria) · **Requirements** (verify requirements are defined and represent stakeholder needs correctly) · **Analysis and Design** (verify the design reflects the requirements) · **Coding/Implementation** (verify the code is efficient and reflects requirements and design) · **Component Testing · Component Integration Testing · System Testing** (**hardware, software, procedures and data representative of production**; **interfaces may be simulated if they give a true representation of performance**) · **System Integration Testing** (**the entire system, representative of production**) · **Acceptance Testing** (**validate against the originally stated user needs and acceptance criteria**).
- **Iterative / incremental:** performance testing **can occur as part of the first iteration, or as an iteration dedicated entirely to performance testing**; **execution may be performed by a separate team**.
- **CI:** most common objective is **regression testing and build stability**; performance tests can join CI **if designed to be executed at a build level**. **Four extra concerns: on-demand test environment (cloud) · which tests to automate, given the short CI timeframe (a subset of more extensive tests run by a specialist team) · creating tests whose main objective is to ensure a change does not negatively impact performance · executing on portions of the system, requiring rapid testing and selectable subsets**.
- **Seven iterative activities: Release Planning** (all iterations in a release; risks identified and assessed; final pre-release testing planned) · **Iteration Planning** (risks assessed **in more detail for each user story**) · **User Story Creation** (**"non-functional" user stories**; criteria in the **acceptance criteria**) · **Design of performance tests · Coding/Implementation** (**component-level testing, e.g., tuning algorithms**) · **Testing/Evaluation** (may be a **separate activity** or a **dedicated iteration**; wider scope for a completed set of stories) · **Delivery** (**monitor performance in the production environment**).

- **COTS / supplier-acquirer:** testing is needed **from both supplier and acquirer perspectives**; **it is often the customer's responsibility to validate that performance meets their requirements**; for **customized vendor-developed software**, criteria **should be specified in the contract**; for **COTS**, **the customer has sole responsibility to test in a realistic test environment prior to deployment**.

2.5 Chapter 3 keyword glossary

The syllabus lists **four keywords** for Chapter 3: **metric, risk, software development lifecycle, test log**. All are examinable at **K1** even where no learning objective names them.

Keyword	Definition to memorise
Metric	A defined, context-qualified measure used to evaluate performance. In Chapter 3 metrics appear at **test completion**, where results are **expressed through metrics which are often aggregated to simplify the meaning of the test results**, and reported via **dashboards**. *(syllabus keyword)*
Risk	A potential problem to product quality — for performance, focused on **time behavior, resource utilization and capacity** — evaluated in terms of **likelihood and impact using clearly defined criteria**. **Business stakeholders influence impact; technical stakeholders influence likelihood.** *(syllabus keyword)*
Software development lifecycle	The model that determines how performance testing activities are **organized and performed** — **sequential**, **iterative and incremental**, or **COTS / supplier-acquirer**. *(syllabus keyword)*
Test log	The record of each test run. Defined in detail in **PTFL-4.4.1**: **date/time of test start · test duration · scripts used (including script mix) and configuration data · test data files used · name and location of data/log files created · HW/SW configuration tested (especially changes between runs) · average and peak CPU and RAM utilization on web and database servers · notes on achieved performance · defects identified.** *(syllabus keyword)*
Residual risk	The risk that remains after mitigation. **It always exists**, and must be communicated as a **realistic and specific evaluation** — ideally as **the percentage of customers likely to experience delays equal to or exceeding certain thresholds**.
Test summary report	The deliverable of **test completion**, in which **performance test results are provided to the stakeholders** such as **architects, managers and product owners**.
Non-functional user story	In Agile, a user story whose **specific performance criteria are described in the associated acceptance criteria**.
Point of test closure	The stage at which **designed tests, test tool assets (test cases and test procedures), test data and other testware are archived or passed on to other testers for later use during system maintenance activities**.

Section 3 — Tables and Summaries

3.1 The architecture risk grid — the three-mark table

Architecture	Memory leaks?	Connectivity?	Signature risk
Single computer	**Yes**	No	**Excessive resource consumption** and **inefficient algorithms that do not use available resources**
Multi-tier	Inherited per server	No	**Poor or non-scalable database design**, **network bottlenecks**, **inadequate bandwidth or capacity on any single server**
Distributed	Inherited	No (not connection-dropping)	**Unreliable or unpredictable remote servers** with **periodic connection problems or intermittent intense load**
Virtualized	**Yes**	No	**Excessive load across all VMs**; **improper host VM configuration → inadequate resources**
Dynamic / cloud-based	**Less of a risk** — the environment **scales to compensate**	No	**Failure to properly configure the self-scaling features during initial setup or subsequent updates**
Client-server / browser-based	**Yes** — much of this code is **written in C and C++**	No	**Connection speed and reliability · public Wi-Fi congestion · firewalls, packet inspection, load balancing**
Mobile	**Yes** — **particularly at risk due to limited available memory**	**Yes**	**Location, battery life, charge state, memory, temperature**; **sensors and radios**; **heavy interaction with local apps and remote web services**
Embedded real-time	**Yes** — languages with **less protection against leaks**	**Yes** — especially **IoT**	**Slowdowns can be catastrophic** (e.g., **vehicle braking**), not merely annoying
Mainframe	**No** — older code in **higher level languages that provide their own memory management**	No	**Now accessible via APIs, web services or the database → unexpected loads affecting throughput of established applications**

The two official multiple-selection answers, memorised

Connectivity risk → mobile and embedded. ("*Mobile and embedded environments are particularly prone to connectivity issues, particularly when embedded software is used in IoT devices.*") **Memory-leak risk → virtualized, client/server and browser-based, mobile, embedded.** (Excluded: **dynamic/cloud-based**, because **the environment will scale to compensate**, and **mainframe**, because **older code is often written in higher level languages that provide their own memory management**.)

3.2 Which activity owns this task?

Task described in a stem	Activity
Risk identification and risk analysis; establishing **the scope of performance testing**; allocating **environments, data, tools and human resources**	**Test planning**

Task described in a stem	Activity
Defining **control measures / action plans**; reacting to **hardware, network or software changes**; **evaluating exit criteria achievement**	**Test monitoring and control**
Analysing the **test basis** (requirements, objectives, SLAs, IT architecture, process models); identifying **load levels, timing conditions and transactions**; **deciding the type of performance test**	**Test analysis**
Designing **modular performance test cases** as **building blocks**	**Test design**
Ordering test cases into performance test procedures; **resetting the environment before each execution**; **establishing production-representative test data**	**Test implementation**
Conducting the test, evaluating results against requirements, **reporting defects**	**Test execution**
Producing the **test summary report**; **aggregating metrics**; using **dashboards**; **archiving testware at test closure**	**Test completion**

3.3 The four-step risk process, with who supplies what

Step	Action	Supplied by
1	**Identify** risks to product quality — **time behavior, resource utilization, capacity**	Both groups
2	**Assess**, addressing the **relevant architecture categories**, evaluating **likelihood and impact** with **clearly defined criteria**	**Impact ← business · likelihood ← technical**
3	**Mitigate** according to the **nature of the risk item and the level of risk**	Both groups
4	**Manage on an ongoing basis**, ensuring risks are **adequately mitigated prior to release**	Both groups

3.4 Sequential lifecycle — phase to activity

Phase	Verb	Activity
Concept	Verify	**Performance goals are defined as acceptance criteria for the project**
Requirements	Verify	**Performance requirements are defined and represent stakeholder needs correctly**
Analysis and Design	Verify	**The system design reflects the performance requirements**
Coding / Implementation	Verify	**The code is efficient and reflects the requirements and design in terms of performance**
Component Testing	Conduct	**Component level performance testing**
Component Integration Testing	Conduct	**Performance testing at the component integration level**
System Testing	Conduct	**System-level testing** with **hardware, software, procedures and data representative of production**; **interfaces may be simulated if they give a true representation of performance**
System Integration Testing	Conduct	**Performance testing with the entire system, representative of the production environment**
Acceptance Testing	**Validate**	**System performance meets the originally stated user needs and acceptance criteria**

3.5 Iterative lifecycle — activity to performance content

Activity	Performance content
Release Planning	**All iterations in a release**; **risks identified and assessed, mitigation planned**; **final pre-release performance testing planned**
Iteration Planning	Testing **within the iteration and as each iteration completes**; **risks assessed in more detail for each user story**
User Story Creation	**Non-functional user stories**, with **performance criteria in the acceptance criteria**
Design of performance tests	**Requirements and criteria from user stories drive test design**
Coding / Implementation	**Component-level performance testing**, e.g., **tuning of algorithms for optimum performance efficiency**
Testing / Evaluation	May be a **separate activity** or a **dedicated performance iteration**; **wider scope for a completed set of user stories** than for a single story
Delivery	**Monitor performance in the production environment** to confirm desired levels **in actual usage**

3.6 CI performance testing — the four extra concerns

Concern	Requirement it creates
Environment setup	**A test environment available on demand, such as a cloud-based performance test environment**
Which tests to automate	A **subset** of more extensive tests, because of **the short timeframe available for CI tests**; the fuller set is run **by a specialist team at other times during an iteration**
Creating the tests	The main objective is **to ensure a change does not negatively impact performance**; **new tests may be required depending on the changes in a build**
Executing on portions of the system	**Tools and environments capable of rapid performance testing**, including **the ability to select subsets of applicable tests**

3.7 One-line answer to every learning objective

LO	If you remember only one sentence
PTFL-3.1.1	Seven activities aligned to the ISTQB test process — and the exam decides them by task: **risk identification is planning**, **ordering test cases into procedures is implementation**.
PTFL-3.2.1	Nine architectures with named risks — **connectivity belongs to mobile and embedded**, **memory leaks to virtualized, client-server, mobile and embedded**, and **C/C++ requires memory monitoring where Python does not**.
PTFL-3.3.1	**Identify, assess, mitigate, manage — started early and repeated regularly throughout the lifecycle**, because **risks change in both impact and likelihood**; residual risk must be **quantified against thresholds**.
PTFL-3.4.1	Sequential = **nine phases, verify then validate**; iterative = **seven activities plus four CI concerns**; COTS = **the customer validates**. The best answer usually **pairs early static review with later dynamic testing**.

Section 4 — Demo Questions with Answers

*Questions 1–7 are the **official ISTQB CT-PT Sample Exam questions for Chapter 3** — questions #11 to #17 — reproduced with their official correct answers and rationale. Questions 8–35 are additional items derived strictly from the syllabus text, in the same style and difficulty. Correct options are marked ✔ — cover the green box and self-test first.*

Part A — The 7 official Chapter 3 questions

Q1. OFFICIAL · Sample Exam #11

When applying the principal performance testing activities, when should risk identification and analysis occur?

a) Test planning ✔

b) Test analysis and design

c) Test implementation and execution

d) Test closure

Correct answer: a | LO: PTFL-3.1.1 (K2) | Syllabus §3.1

Why: (a) Risk identification and analysis is done at the test planning stage. This information is then used for the analysis and design of the tests. (b), (c) and (d) are incorrect because **this is too late in the process**. The syllabus places this explicitly: "***During test planning, risk identification and risk analysis activities are completed*** *and relevant information is updated in any test planning documentation (e.g., test plan, level test plan).*"

Q2. OFFICIAL · Sample Exam #12

When applying the principal performance testing activities, when should the test cases be ordered into performance test procedures?

a) Test planning

b) Test analysis and design

c) Test implementation and execution ✔

d) Test closure

Correct answer: c | LO: PTFL-3.1.1 (K2) | Syllabus §3.1

Why: (c) This should occur during the test implementation and execution activities. (a) and (b) are incorrect because **this is when the test cases are planned and designed**. (d) is incorrect because **this is too late**. The syllabus wording is: "*In the **implementation phase**, **performance test cases are ordered into performance test procedures**. These performance test procedures should **reflect the steps normally taken by the user** and other functional activities that are to be covered during performance testing.*"

Q3. OFFICIAL · Sample Exam #13 · multiple selection

*Consider the following technical environments: **1.** Single computer · **2.** Multi-tier system · **3.** Distributed · **4.** Virtualized · **5.** Dynamic/Cloud-based · **6.** Client/Server and Browser-based · **7.** Mobile · **8.** Embedded · **9.** Mainframe*

Which of these is most likely to have a performance risk due to connectivity issues?

a) 2, 3

b) 7, 8 ✔

c) 5, 6, 7, 9

d) 2, 4, 5, 8

Correct answer: b | LO: PTFL-3.2.1 (K2) | Syllabus §3.2

Why: (b) 7 and 8 — mobile and embedded. The official rationale: **mobile and embedded environments are particularly prone to connectivity issues, particularly when embedded software is used in the IoT devices**. (a) and (d) are incorrect because **these environments are not prone to connection dropping**. (c) is incorrect because **other than #7, the other environments are not prone to connection dropping**. The syllabus text supports this directly: mobile applications face **limited and variable resources and connectivity**, and embedded real-time systems have **many of the risks of mobile devices, including (increasingly)**

connectivity-related issues since these devices are connected to the Internet.

Q4. OFFICIAL · Sample Exam #14 · multiple selection

Consider the following technical environments: ***1.*** *Virtualized* · ***2.*** *Dynamic/Cloud-based* · ***3.*** *Client/Server and Browser-based* · ***4.*** *Mobile* · ***5.*** *Embedded* · ***6.*** *Mainframe*

Which of these is most likely to have a performance risk due to memory leaks?

a) 1, 2, 3, 6

b) 2, 3, 4, 5

c) 1, 2, 4, 6

d) 1, 3, 4, 5 ✔

Correct answer: d | LO: PTFL-3.2.1 (K2) | Syllabus §3.2

Why: (d) 1, 3, 4, 5 — virtualized, client/server and browser-based, mobile, embedded. The official rationale gives a reason per environment: **the virtualized environment uses shared resources across different applications, including memory. The dynamic/cloud-based environment is designed to dynamically scale, so while memory leaks may occur, they are less of a risk because the environment will scale to compensate. Client/Server and browser-based environments are prone to memory leaks, particularly since a lot of this code is written in C and C++. Mobile environments are prone to leakage and are particularly at risk due to the limited available memory. The same is true of the embedded environments (and these also tend to be written in languages that have less protection against leaks). Mainframes tend to run older code, often written in the higher level languages that provide their own memory management.**

Q5. OFFICIAL · Sample Exam #15

If performance testing is being conducted for software that is written in C++, what do you need to monitor that wouldn't be a concern if the software was written in Python?

a) Memory use ✔

b) Network latency

c) Connectivity

d) Batch processing

Correct answer: a | LO: PTFL-3.2.1 (K2) | Syllabus §3.2

Why: (a) C and C++ require the programmer to do their own memory management, which sometimes results in memory leaks (where memory is allocated, but not de-allocated). (b), (c) and (d) **would have the same issues regardless of the language used.** The syllabus supports this in its closing paragraph on non-architectural technical decisions: **memory leaks are more common with languages that allow direct heap memory management, such as C and C++**.

Q6. OFFICIAL · Sample Exam #16 · K4 scenario

You are working on a project that tracks health history information for patients across a region. The number of records handled by the system is in the millions due to the large number of patients in the region. Patient information must be accessible to doctors in offices, hospitals and urgent care facilities. The information should be presented to the requestor within three seconds of request, particularly for patients with critical allergies and preconditions.

Given this information, when is the best time in the project to analyze and assess the performance risks?

a) During the requirements phase and again just prior to executing the performance tests

b) After design but prior to coding

c) During system testing and again prior to the performance tests

d) Repeatedly throughout the requirements, development and performance testing ✔

Correct answer: d | LO: PTFL-3.3.1 (K4) | Syllabus §3.3

Why: (d) Since performance is critical, the risks must be analyzed as early as possible and then repeatedly as the system is assembled since risks may change both in impact and likelihood. (a) is not correct because **it ignores looking at the risks during development**. (b) is not correct because **it leaves out the requirements and development and at the completion of development**. (c) is not correct because **it leaves out the requirements and development options for analysis**. The syllabus principle: the risk analysis process must be **started early and repeated regularly**, with **an iterative approach to performance risk identification, assessment, mitigation, and management throughout the software development lifecycle**.

Q7. OFFICIAL · Sample Exam #17 · K4 scenario

You are working on a project that tracks health history information for patients across a region. The number of records handled by the system is in the millions due to the large number of patients in the region. Patient information must be accessible to doctors in offices, hospitals and urgent care facilities. The information should be presented to the requestor within three seconds of request, particularly for patients with critical allergies and preconditions.

What is the best way to address the performance testing for the response time?

a) Test from the UI with the full data set loaded to ensure the response time will be adequate when the database is loaded

b) Test via the web services at the API level to ensure access to the data is fast enough without having the testing complicated by the UI

c) Conduct a technical review of the database implementation and conduct a performance test from the UI with the full database loaded ✔

d) Conduct a network assessment to ensure there are no latency issues between the database server and the web servers, then test with a network scanner to ensure no collisions are occurring that might cause performance delays

Correct answer: c | LO: PTFL-3.4.1 (K4) | Syllabus §3.4

Why: (c) The best way to do the testing is to first do a technical analysis because this can be done at design time. Once implemented, full testing will be needed to verify that the solution does indeed deliver the performance required. (a) is not correct because **this leaves out the static testing**. (b) is not correct because **this would bypass any issues that might occur at the UI end and doesn't consider the database loading**. (d) is not correct because **the network is less likely an issue regarding the data handling for such a large amount of data**. **The answer pattern: pair an early static review with a later dynamic test.**

Part B — 28 additional syllabus-derived questions

Q8. Recall

Why does the syllabus say test planning is particularly important for performance testing?

a) Because the test plan is a contractual deliverable in most projects

b) Because of the need for the allocation of test environments, test data, tools and human resources ✔

c) Because performance test cases cannot be designed without a plan

d) Because exit criteria are evaluated during planning

Correct answer: b | LO: PTFL-3.1.1 (K2) | Syllabus §3.1

Why: "*Test planning is* ***particularly important for performance testing due to the need for the allocation of test environments, test data, tools and human resources***. *In addition,* ***this is the activity in which the scope of performance testing is established***." Option (d) belongs to **test monitoring and control**, where **the performance test objectives are evaluated to check for exit criteria achievement**.

Q9. Activity mapping

During which activity are the required type(s) of performance test — for example load, stress or scalability — decided?

a) Test planning

b) Test design

c) Test analysis ✔

d) Test implementation

Correct answer: c | LO: PTFL-3.1.1 (K2) | Syllabus §3.1

Why: Test analysis. "*Specific test conditions are identified such as* ***load levels, timing conditions, and transactions to be tested***. ***The required type(s) of performance test (e.g., load, stress, scalability) are then decided.***" Test analysis is also where the **test basis** — performance requirements, test objectives, **SLAs**, **IT architecture**, **process models** — is analysed, possibly with the support of **spreadsheets or capacity planning tools**.

Q10. Activity mapping

Which activity includes defining control measures to provide action plans should the infrastructure fail to generate the desired loads?

a) Test monitoring and control ✔

b) Test planning

c) Test execution

d) Test completion

Correct answer: a | LO: PTFL-3.1.1 (K2) | Syllabus §3.1

Why: Test monitoring and control. The syllabus lists four such control measures: **increasing the load generation capacity if the infrastructure does not generate the desired loads as planned · changed, new or replaced hardware · changes to network components · changes to software implementation**. It also states that **the performance test objectives are evaluated to check for exit criteria achievement** in this activity.

Q11. Recall

How does the syllabus say performance test cases are generally created during test design?

a) As a single end-to-end script per business process

b) As recorded protocol traces without parameterization

c) As one test case per operational profile, never combined

d) In modular form, so that they may be used as the building blocks of larger, more complex performance tests ✔

Correct answer: d | LO: PTFL-3.1.1 (K2) | Syllabus §3.1

Why: "*Performance test cases are designed. These are **generally created in modular form** so that they may be used as the **building blocks of larger, more complex performance tests** (see section 4.2).*" Modularity is what later allows **scripts for separate operational profiles to be combined into a scenario implementing the whole load profile** (**PTFL-4.2.7**).

Q12. Recall

Because performance testing is typically data-driven, what process is needed during test implementation?

a) A process to anonymise all production data before every run

b) A process to establish test data that is representative of actual production data in volume and type so that production use can be simulated ✔

c) A process to generate synthetic data only, so no production data is ever used

d) A process to archive test data at the point of test closure

Correct answer: b | LO: PTFL-3.1.1 (K2) | Syllabus §3.1

Why: "*Since performance testing is **typically data-driven**, a process is needed to **establish test data that is representative of actual production data in volume and type so that production use can be simulated**.*" Test implementation also covers **establishing and/or resetting the test environment before each test execution**. Archiving testware belongs to **test completion**.

Q13. Recall

At the point of test closure, what happens to the testware?

a) It is deleted to prevent reuse of obsolete scripts

b) It is handed to the vendor under the supplier/acquirer model

c) Designed tests, test tool assets, test data and other testware are archived or passed on to other testers for later use during system maintenance activities ✔

d) It is converted into non-functional user stories for the next release

Correct answer: c | LO: PTFL-3.1.1 (K2) | Syllabus §3.1

Why: "*At the close of a defined period of performance testing, a **point of test closure** may be reached where **designed tests, test tool assets (test cases and test procedures), test data and other testware are archived or passed on to other testers for later use during system maintenance activities**.*" This sits within **test completion**, alongside the **test summary report**, **aggregated metrics** and **dashboards**.

Q14. Tricky

Which statement best captures the syllabus's description of the nature of performance testing?

a) It is iterative — information from one test corrects or optimizes application and system parameters, and the next iteration shows the results of the modifications, until test objectives are reached ✔

b) It is a single pass conducted once the system is complete

c) It is a strictly sequential process that cannot revisit earlier activities

d) It concludes when the first set of measurements is captured

Correct answer: a | LO: PTFL-3.1.1 (K2) | Syllabus §3.1

Why: "*Performance testing is **iterative in nature**. **Each test provides valuable insights** into application and system performance. The information gathered from one test is used to **correct or optimize application and system parameters**. The next test iteration will then **show the results of modifications**, and so on **until test objectives are reached**.*" This echoes Chapter 1's description of performance testing as **experimentation** conducted **iteratively**.

Q15. Architecture risk

Which performance risks does the syllabus associate specifically with virtualized systems?

a) Poor or non-scalable database design and network bottlenecks

b) Firewalls, packet inspection and server load balancing

c) Excessive load on the hardware across all the virtual machines, and improper configuration of the host virtual machine resulting in inadequate resources ✔

d) Unexpected loads arriving through newly exposed APIs and web services

Correct answer: c | LO: PTFL-3.2.1 (K2) | Syllabus §3.2

Why: Those two are the only risks the syllabus names as **arising specifically from virtualization**. (a) belongs to **multi-tier systems**, (b) to **client-server systems**, and (d) to **mainframe applications**. Note the link back to **PTFL-2.1.1**, where **background load on shared resources** is called out as **especially relevant in virtualized environments**.

Q16. Architecture risk

What performance risk does the syllabus associate with dynamic / cloud-based systems?

a) They cannot scale beyond a fixed hardware ceiling

b) Failures to properly configure the self-scaling features during initial setup or subsequent updates ✔

c) They are the most prone of all architectures to memory leaks

d) They cannot be monitored in production

Correct answer: b | LO: PTFL-3.2.1 (K2) | Syllabus §3.2

Why: Dynamic/cloud-based systems **offer the ability to scale on demand** and are **typically distributed and virtualized multi-tier systems, albeit with self-scaling features designed specifically to mitigate some of the performance risks associated with those architectures**. **However, there are risks associated with failures to properly configure these features during initial setup or subsequent updates.** On memory leaks, the official rationale is the opposite of (c): they are **less of a risk because the environment will scale to compensate**.

Q17. Architecture risk

Which risks does the syllabus name for client-server systems, over and above the single-computer risks that apply to the client-side code?

a) Battery life, charge state and device temperature

b) Disk fragmentation and operating system mismanagement

c) Unexpected loads from newly exposed APIs

d) Connection speed and reliability issues, network congestion at the client connection point such as public Wi-Fi, and potential problems due to firewalls, packet inspection and server load balancing ✔

Correct answer: d | LO: PTFL-3.2.1 (K2) | Syllabus §3.2

Why: The syllabus states that **since there is code running on the client, the single computer risks apply to that code, while the server-side issues mentioned above apply as well**, and then adds **connection speed and reliability issues, network congestion at the client connection point (e.g., public Wi-Fi), and potential problems due to firewalls, packet inspection and server load balancing**. (a) belongs to **mobile applications**; (b) to **single computer systems**; (c) to **mainframe applications**.

Q18. Architecture risk

Which factors does the syllabus list as affecting the limited and variable resources and connectivity available on a mobile device?

a) Location, battery life, charge state, available memory on the device and temperature ✔

b) Firewall configuration, packet inspection and load balancing

c) Disk fragmentation and background security software

d) Host virtual machine configuration and hardware oversubscription

Correct answer: a | LO: PTFL-3.2.1 (K2) | Syllabus §3.2

Why: The syllabus lists exactly these five. It adds that for apps using **device sensors or radios such as accelerometers or Bluetooth**, **slow dataflows from those sources could create problems**, and that **mobile applications often have heavy interactions with other local mobile apps and remote web services, any of which can potentially become a performance efficiency bottleneck**.

Q19. Tricky · contrast

How does the syllabus distinguish embedded real-time systems from mobile devices in terms of performance risk?

a) Embedded systems are never connected to the Internet, so connectivity is not a risk

b) Embedded systems have unlimited memory compared with mobile devices

c) They share many risks including connectivity, but the diminished performance of a mobile video game is usually not a safety hazard, while such slowdowns in a vehicle braking system could prove catastrophic ✔

d) Embedded systems are exempt from resource utilization risks because they are purpose-built

Correct answer: c | LO: PTFL-3.2.1 (K2) | Syllabus §3.2

Why: The syllabus says embedded real-time systems **often have many of the risks of mobile devices, including (increasingly) connectivity-related issues since these devices are connected to the Internet**, and then draws the distinction in exactly the terms of option (c). Examples given of embedded real-time systems are **cars (entertainment systems and intelligent braking systems), elevators, traffic signals and HVAC systems**.

Q20. Architecture risk

What does the syllabus identify as the modern performance risk for mainframe applications?

a) They cannot process batch workloads within a nightly window

b) Many are now accessible via APIs, web services, or through their database, which can result in unexpected loads that affect throughput of established applications ✔

c) They are the architecture most prone to memory leaks

d) They cannot be virtualized

Correct answer: b | LO: PTFL-3.2.1 (K2) | Syllabus §3.2

Why: "***Most are quite predictable and fast when used as originally designed***, *but* ***many of these are now accessible via APIs, web services, or through their database, which can result in unexpected loads that affect throughput of established applications***." On memory leaks, the official rationale excludes mainframes: **mainframes tend to run older code, often written in the higher level languages that provide their own memory management**.

Q21. Tricky · combination

What does the syllabus say about applications that incorporate more than one of the nine architectures?

a) Only the risks of the dominant architecture apply

b) The risks cancel each other out when architectures are combined

c) Combined architectures are outside the scope of performance testing

d) All relevant risks will apply — and with IoT and mobile it is possible that all architectures are present in some form and thus all risks can apply ✔

Correct answer: d | LO: PTFL-3.2.1 (K2) | Syllabus §3.2

Why: "*Note that **any particular application or system may incorporate two or more of the architectures listed above**, which means that **all relevant risks will apply to that application or system**. In fact, given the **Internet of Things and the explosion of mobile applications** — two areas where **extreme levels of interaction and connection is the rule — it is possible that all architectures are present in some form in an application, and thus all risks can apply**.*"

Q22. Non-architectural risk

Besides architecture, which other technical decisions does the syllabus say influence and create performance risks?

a) Choice of test tool vendor and licence model

b) Choice of test level and test type

c) Direct heap memory management in languages such as C and C++, relational versus non-relational databases, and the choice of a recursive as opposed to an iterative algorithm ✔

d) Choice of operational profile and load profile

Correct answer: c | LO: PTFL-3.2.1 (K2) | Syllabus §3.2

Why: The syllabus gives these three examples and notes that such decisions **extend all the way down to the design of individual functions or methods**. It closes with a caution: **as a tester, the ability to know about or even influence such decisions will vary, depending on the roles and responsibilities of testers within the organization and software development lifecycle**.

Q23. Architecture risk

Which architecture does the syllabus define as running entirely on one non-virtualized computer, and what two categories of risk does it name for it?

a) Single computer systems; excessive resource consumption, and inefficient implementation of algorithms that do not make use of available resources ✔

b) Client-server systems; connection speed and firewall interference

c) Multi-tier systems; database design and network bottlenecks

d) Virtualized systems; hardware oversubscription and host misconfiguration

Correct answer: a | LO: PTFL-3.2.1 (K2) | Syllabus §3.2

Why: Single computer systems "*run **entirely on one non-virtualized computer***". Performance can degrade due to **excessive resource consumption including memory leaks, background activities such as security software, slow storage subsystems (e.g., low-speed external devices or disk fragmentation), and operating system mismanagement**; and due to **inefficient implementation of algorithms which do not make use of available resources (e.g., main memory) and as a result execute slower than required**.

Q24. Tricky · distributed

What distinguishes a distributed system from a multi-tier system in the syllabus's treatment, and what extra risk follows?

a) Distributed systems run on a single server; the extra risk is disk fragmentation

b) The various servers may change dynamically; the extra risk is critical workflows or dataflows to, from, or through unreliable or unpredictable remote servers, especially with periodic connection problems or intermittent intense load ✔

c) Distributed systems are always virtualized; the extra risk is host misconfiguration

d) Distributed systems are always cloud-based; the extra risk is self-scaling misconfiguration

Correct answer: b | LO: PTFL-3.2.1 (K2) | Syllabus §3.2

Why: "*These are systems of systems, **similar to a multi-tier architecture, but the various servers may change dynamically** — such as an e-commerce system that accesses **different inventory databases depending on the geographic location of the person placing the order**. **In addition to the risks associated with multi-tier architectures**, this architecture can experience performance problems due to **critical workflows or dataflows to, from, or through unreliable or unpredictable remote servers**, especially when such servers suffer **periodic connection problems or intermittent periods of intense load**.*"

Q25. K4 · process

In the syllabus's four-step process for performance-related product quality risks, what happens at step 2?

a) Risks are identified, focusing on time behavior, resource utilization and capacity

b) Mitigation actions are taken based on the nature of the risk item and the level of risk

c) Risks are assessed, ensuring the relevant architecture categories are addressed, and the overall level of risk is evaluated in terms of likelihood and impact using clearly defined criteria ✔

d) Risks are managed on an ongoing basis to ensure adequate mitigation prior to release

Correct answer: c | LO: PTFL-3.3.1 (K4) | Syllabus §3.3

Why: The four steps are **identify → assess → mitigate → manage**, and step 2 is exactly as stated in (c). Note the explicit cross-reference: assessment must ensure **the relevant architecture categories (see Section 3.2) are addressed** — so **PTFL-3.2.1 knowledge is a prerequisite for the K4 question**.

Q26. K4 · stakeholders

According to the syllabus, what must business stakeholders appreciate about performance risk?

a) That intended usage, business-, societal- or safety-criticality, potential financial and/or reputational damage, and civil or criminal legal liability create risks and influence the impact of failures ✔

b) That architecture, design and implementation decisions influence the likelihood of defects

c) That performance testing should be conducted only in production

d) That residual risk can be eliminated entirely with sufficient mitigation

Correct answer: a | LO: PTFL-3.3.1 (K4) | Syllabus §3.3

Why: The syllabus splits the two groups precisely: **business stakeholders must appreciate that intended usage, business-, societal-, or safety-criticality, potential financial and/or reputational damage, civil or criminal legal liability and similar factors affect risk from a business perspective, creating risks and influencing the impact of failures**, while **technical stakeholders must appreciate that architecture, design, and implementation decisions affect performance risks from a technical perspective, creating risks and influencing the likelihood of defects**. **Business = impact. Technical = likelihood.**

Q27. K4 · residual risk

At the end of a performance test campaign, the test manager reports to the steering group: "Yes, it's still possible for customers to experience long delays during check out."

Why does the syllabus consider this an inadequate statement of residual risk, and what would be better?

a) It is inadequate because residual risk should never be reported to business stakeholders

b) It is inadequate because performance risks can be eliminated completely with enough mitigation

c) It is adequate as stated; no improvement is required

d) It gives no idea of what amount of risk mitigation has occurred or of the level of risk that remains — providing clear insight into the percentage of customers likely to experience delays equal to or exceeding certain thresholds would help people understand the status ✔

Correct answer: d | LO: PTFL-3.3.1 (K4) | Syllabus §3.3

Why: This is the syllabus's own worked contrast, quoted almost verbatim. It follows from the principle that **performance-related risks can never be avoided completely — some risk of performance-related production failure will always exist** — so the risk management process **must include providing a realistic and specific evaluation of the residual level of risk to the business and technical stakeholders involved**.

Q28. K4 · early mitigation

On a new product, the expected number of transactions and simultaneous users genuinely cannot be specified early in the project.

What does the syllabus recommend?

a) Defer all performance risk analysis until the numbers are known

b) Ensure design and architecture decisions allow for highly variable scalability, for example on-demand cloud-based computing resources, so that early risk mitigation decisions can be made ✔

c) Test in production, since only real usage will reveal the numbers

d) Set arbitrary targets and treat them as contractual acceptance criteria

Correct answer: b | LO: PTFL-3.3.1 (K4) | Syllabus §3.3

Why: "*When critical performance-related decisions such as **the expected number of transactions or simultaneous users cannot be specified early in the project**, it is important that **design and architecture decisions allow for highly variable scalability (e.g., on-demand cloud-based computing resources)**. **This enables early risk mitigation decisions to be made.***" It is a direct illustration of the principle that **risk mitigation and management must span and influence the entire software development process, not just dynamic testing**.

Q29. K4 · static prediction

A system will handle large volumes of data via a relational database. The team is concerned about the slow performance of many-to-many joins caused by poor database design.

According to the syllabus, when would this problem typically reveal itself, and how could it be predicted earlier?

a) It reveals itself during unit testing and can be predicted by profiling

b) It reveals itself only in production and cannot be predicted

c) It may only reveal itself during dynamic testing with large-scale test datasets such as those used during system test, but a careful technical review including experienced database engineers can predict the problems prior to database implementation ✔

d) It reveals itself during acceptance testing and can be predicted by crowd testing

Correct answer: c | LO: PTFL-3.3.1 (K4) | Syllabus §3.3

Why: This is the syllabus's own worked example, and it is the reasoning that produces the official answer to question #17 — **pair an early static review with a later dynamic test**. The syllabus adds: **after such a review, in an iterative approach, risks are identified and assessed again**.

Q30. K4 · timing

What does the syllabus warn testers against, when it comes to the timing of performance risk work?

a) Relying entirely on performance testing conducted towards the end of the system test level and system integration test level ✔

b) Beginning risk analysis before the architecture has been chosen

c) Involving business stakeholders in technical risk assessment

d) Repeating the risk analysis more than once per release

Correct answer: a | LO: PTFL-3.3.1 (K4) | Syllabus §3.3

Why: "*For performance-related risks, it is especially important that* ***the risk analysis process be started early and is repeated regularly***. *In other words,* ***the tester should avoid relying entirely on performance testing conducted towards the end of the system test level and system integration test level***." The syllabus notes that **larger and more complex systems of systems projects** have met **unfortunate surprises** for precisely this reason.

Q31. K4 · sequential

In a sequential development model, what does the syllabus describe as ideal practice for performance testing?

a) Scheduling all performance testing into a dedicated phase after system integration testing

b) Including performance criteria as part of the acceptance criteria defined at the outset of the project, with performance testing activities conducted throughout the lifecycle ✔

c) Deferring performance criteria until the acceptance testing phase, when user needs are best understood

d) Contracting performance testing to the vendor under a supplier/acquirer model

Correct answer: b | LO: PTFL-3.4.1 (K4) | Syllabus §3.4

Why: "*The* ***ideal practice*** *of performance testing in sequential development models is to* ***include performance criteria as a part of the acceptance criteria which are defined at the outset of a project***. *Reinforcing the lifecycle view of testing,* ***performance testing activities should be conducted throughout the software development lifecycle***. *As the project progresses,* ***each successive performance test activity should be based on items defined in the prior activities***."

Q32. Recall · sequential

At which sequential phase does the syllabus permit system interfaces to be simulated, and under what condition?

a) Component integration testing, provided the simulator is built by the same team

b) Acceptance testing, provided the customer agrees

c) System integration testing, provided the whole system is otherwise representative

d) System testing, provided the simulated interfaces give a true representation of performance ✔

Correct answer: d | LO: PTFL-3.4.1 (K4) | Syllabus §3.4

Why: At **System Testing**: "*Conduct performance testing at the system level, which includes* ***hardware, software, procedures and data that are representative of the production environment***. ***System interfaces may be simulated provided that they give a true representation of performance.***" At **System Integration Testing** the requirement is stricter: **conduct performance testing with the entire system which is representative of the production environment**.

Q33. Recall · CI

What does the syllabus give as the main objective of performance tests included in Continuous Integration?

a) To determine the maximum capacity of the production environment

b) To replace the specialist team's more extensive performance testing

c) To ensure a change does not negatively impact performance ✔

d) To validate the operational profiles used in the load profile

Correct answer: c | LO: PTFL-3.4.1 (K4) | Syllabus §3.4

Why: "***The main objective of performance tests as part of CI is to ensure a change does not negatively impact performance.*** *Depending on the changes made for any given build,* ***new performance tests may be required***." The syllabus also notes that CI performance tests **may be a subset of more extensive performance tests that are conducted by a specialist team at other times during an iteration**, because of **the short timeframe available for CI tests** — so (b) is explicitly wrong.

Q34. Recall · Agile

In Agile methodologies, where does the syllabus say the specific performance criteria are described?

a) In the acceptance criteria associated with "non-functional" user stories ✔

b) In the performance test plan only, never in user stories

c) In the definition of done for the release

d) In the contract between vendor and customer

Correct answer: a | LO: PTFL-3.4.1 (K4) | Syllabus §3.4

Why: "***User stories often form the basis of performance requirements in Agile methodologies***, *with the* ***specific performance criteria described in the associated acceptance criteria***. *These are referred to as* ***"non-functional" user stories***." Option (d) describes the **customized vendor-developed software** case under the **supplier/acquirer** model.

Q35. K4 · COTS

An organisation is buying a commercial off-the-shelf application rather than developing it, and asks who is responsible for confirming that it performs adequately.

What does the syllabus say?

a) The vendor has sole responsibility, since they developed the product

b) The customer has sole responsibility to test the performance of the product in a realistic test environment prior to deployment ✔

c) Responsibility is shared equally and defined by the contract

d) No performance testing is required for COTS products

Correct answer: b | LO: PTFL-3.4.1 (K4) | Syllabus §3.4

Why: "*In the case of* ***COTS applications***, ***the customer has sole responsibility to test the performance of the product in a realistic test environment prior to deployment***." Contrast with **customized vendor-developed software**, where **performance requirements and associated acceptance criteria should be specified as part of the contract between the vendor and customer** — and with the general rule that **regardless of the source of the application, it is often the responsibility of the customer to validate that the performance meets their requirements**.

Most Important Questions From This Chapter

Ranking rationale: the ISTQB Exam Structure Table gives Chapter 3 ***seven questions across four objectives****, with* ***PTFL-3.2 receiving three*** *and* ***PTFL-3.1 receiving two****. The two K4 objectives get one each but are the first analysis questions on the paper.*

Rank	Topic / question focus	LO	Why it matters
1	The nine architectures and their named risks — especially **connectivity** and **memory leaks** — Q3, Q4, Q5, Q15–Q24	PTFL-3.2.1	**3 guaranteed marks — 43% of the chapter.** Two of the three are multiple-selection items decided by one table.
2	Which of the seven activities owns a named task — Q1, Q2, Q8–Q14	PTFL-3.1.1	**2 guaranteed marks.** Both official questions name a task and ask for the activity; **planning owns risk analysis, implementation owns procedures**.
3	When to analyse and assess performance risks across the lifecycle — Q6, Q25–Q30	PTFL-3.3.1	**1 guaranteed K4 mark.** The answer spans **the whole lifecycle**; every distractor omits a phase.
4	Choosing the right combination of activities for a project — Q7, Q31–Q35	PTFL-3.4.1	**1 guaranteed K4 mark.** The winning pattern is **early static review plus later dynamic test**.
5	The non-architectural technical decisions — **C/C++ heap management**, relational vs non-relational, recursive vs iterative — Q5, Q22	PTFL-3.2.1	One of the three PTFL-3.2 marks was officially spent here, not on an architecture list at all.

Section 5 — Exam Tricks

5.1 Frequently tested topics (evidence-based)

The ISTQB Exam Structure Table for CT-PT allocates Chapter 3 **seven questions**. **PTFL-3.2 receives three** and **PTFL-3.1 receives two**; the two **K4** objectives receive one each. There is **no selectable group**, so all four objectives appear on every paper.

LO	K	Questions	Points	Sample Exam #	Format used officially
PTFL-3.1	K2	**2**	**2**	#11, #12	"**When** should <task> occur?" — name the activity that owns a task
PTFL-3.2	K2	**3**	**3**	#13, #14, #15	Two **multiple-selection** items ("which of these environments…") plus one **language** question
PTFL-3.3	**K4**	**1**	**1**	#16	Scenario — "**when** is the best time to analyze and assess the performance risks?"
PTFL-3.4	**K4**	**1**	**1**	#17	Scenario — "what is the **best way** to address the performance testing for the response time?"
Chapter 3 total	—	**7**	**7**	**#11–#17**	**17.5% of the paper · the first two K4 questions · 195 minutes of teaching**

> **The arithmetic before you sit the exam**
>
> CT-PT is **40 questions worth 40 points in 90 minutes** (**113 minutes** with the 25% non-native-language allowance); the pass mark is **65% = 26 points**. Chapter allocation: **Ch1 = 6 · Ch2 = 4 · Ch3 = 7 · Ch4 = 20 · Ch5 = 3**. Chapter 3 delivers **7 marks for 195 minutes — 2.2 marks per teaching hour**, the **lowest ratio in the syllabus**. Do not over-invest here at the expense of Chapter 4, which is worth nearly three times as much.

5.2 Common distractors observed

- **Risk identification placed in test analysis.** It belongs to **test planning**; **test analysis** analyses the **test basis** and decides **load levels, timing conditions, transactions and the type of performance test**.
- **Ordering test cases into procedures placed in test design.** Design creates **modular test cases**; **implementation orders them into procedures**.
- **Distributed or multi-tier offered for a connectivity question.** The official rationale is blunt: **these environments are not prone to connection dropping**. Connectivity means **mobile and embedded**.
- **Dynamic/cloud-based included in a memory-leak list.** Leaks may occur, but **the environment will scale to compensate**, so it is **less of a risk**.
- **Mainframe included in a memory-leak list. Older code, often written in higher level languages that provide their own memory management.**
- **Network latency or connectivity offered as the C++ versus Python difference.** They **would have the same issues regardless of the language used**. The answer is **memory use**.
- **A risk-timing option that names only some phases.** The K4 answer spans **requirements, development and performance testing — repeatedly**.
- **A single-technique answer to the K4 lifecycle question.** UI-only **leaves out the static testing**; API-only **bypasses UI issues and ignores database loading**; network-only is wrong because **the network is less likely an issue for such a large amount of data**.
- **Residual risk stated qualitatively.** **"It's still possible for customers to experience long delays" gives no idea of what amount of risk mitigation has occurred or of the level of risk that remains**.
- **Simulation permitted at the wrong level. System testing** allows simulated interfaces **provided that they give a true representation of performance**; **system integration testing** requires **the entire system**.

- **CI performance tests described as replacing the specialist team's tests.** They are a **subset**, because of **the short timeframe available for CI tests**.
- **The vendor made responsible for COTS performance.** For **COTS**, **the customer has sole responsibility to test the performance of the product in a realistic test environment prior to deployment**.

5.3 Keyword recognition (map words → answers)

Keyword in the stem	Points toward
'risk identification', 'risk analysis', 'scope of performance testing', 'allocation of environments, data, tools and human resources'	**Test planning** (PTFL-3.1.1)
'control measures', 'action plans', 'exit criteria achievement', 'increasing the load generation capacity'	**Test monitoring and control** (PTFL-3.1.1)
'test basis', 'SLAs', 'IT architecture', 'process models', 'load levels, timing conditions and transactions', 'capacity planning tools'	**Test analysis** (PTFL-3.1.1)
'modular form', 'building blocks of larger, more complex performance tests'	**Test design** (PTFL-3.1.1)
'ordered into performance test procedures', 'resetting the test environment', 'data-driven', 'representative of actual production data in volume and type'	**Test implementation** (PTFL-3.1.1)
'conducted, often by using performance test tools', 'defects are reported'	**Test execution** (PTFL-3.1.1)
'test summary report', 'architects, managers, product owners', 'dashboards', 'archived or passed on', 'point of test closure'	**Test completion** (PTFL-3.1.1)
'entirely on one non-virtualized computer', 'background activities such as security software', 'disk fragmentation', 'operating system mismanagement'	**Single computer systems** (PTFL-3.2.1)
'database server, application server, presentation server', 'non-scalable database design', 'network bottlenecks'	**Multi-tier systems** (PTFL-3.2.1)
'servers may change dynamically', 'unreliable or unpredictable remote servers', 'geographic location'	**Distributed systems** (PTFL-3.2.1)
'physical hardware hosts multiple virtual computers', 'host virtual machine', 'excessive load across all the virtual machines'	**Virtualized systems** (PTFL-3.2.1)
'scale on demand', 'self-scaling features', 'initial setup or subsequent updates'	**Dynamic / cloud-based systems** (PTFL-3.2.1)
'public Wi-Fi', 'firewalls', 'packet inspection', 'server load balancing'	**Client-server systems** (PTFL-3.2.1)
'battery life', 'charge state', 'temperature', 'accelerometers', 'Bluetooth'	**Mobile applications** (PTFL-3.2.1)
'elevators', 'traffic signals', 'HVAC', 'intelligent braking systems', 'catastrophic'	**Embedded real-time systems** (PTFL-3.2.1)
'decades-old', 'data center', 'now accessible via APIs, web services, or through their database'	**Mainframe applications** (PTFL-3.2.1)
'C and C++', 'direct heap memory management', 'allocated but not de-allocated'	**Memory use monitoring** — the non-architectural risk (PTFL-3.2.1)
'likelihood and impact', 'clearly defined criteria', 'adequately mitigated prior to release'	**The four-step risk process** (PTFL-3.3.1)

Keyword in the stem	Points toward
'financial and/or reputational damage', 'civil or criminal legal liability', 'safety-criticality'	**Business stakeholders — impact** (PTFL-3.3.1)
'architecture, design, and implementation decisions'	**Technical stakeholders — likelihood** (PTFL-3.3.1)
'many-to-many joins', 'experienced database engineers', 'prior to database implementation'	**Static review predicts what dynamic testing would only reveal late** (PTFL-3.3.1)
'percentage of customers likely to experience delays equal to or exceeding certain thresholds'	**Residual risk stated well** (PTFL-3.3.1)
'acceptance criteria defined at the outset', 'each successive activity based on prior activities'	**Sequential ideal practice** (PTFL-3.4.1)
'interfaces may be simulated provided they give a true representation of performance'	**System testing** (PTFL-3.4.1)
'available on demand, such as a cloud-based performance test environment', 'subset of more extensive tests', 'ensure a change does not negatively impact performance'	**CI concerns** (PTFL-3.4.1)
'non-functional user stories', 'tuning of algorithms', 'dedicated iteration for performance testing'	**Iterative lifecycle activities** (PTFL-3.4.1)
'COTS', 'sole responsibility', 'realistic test environment prior to deployment'	**Supplier/acquirer models** (PTFL-3.4.1)

5.4 Typical traps

- **Multiple-selection questions are decided by exclusions, not inclusions.** For the memory-leak item, the fastest route is to strike **dynamic/cloud-based** (**scales to compensate**) and **mainframe** (**higher level languages with their own memory management**). What remains is the answer.
- **Two activities are offered as one option.** "Test analysis and design" and "test implementation and execution" appear as single options. Read them as pairs and decide whether the task belongs to **either** half.
- **The K4 questions share a scenario with Chapter 4.** Questions #16 and #17 use the **health-records** scenario that returns for **#18–#21, #26–#28, #31 and #37**. Read it once, carefully, and note the two hard facts: **millions of records** and **three seconds, particularly for critical allergies and preconditions**.
- **"Best way" questions reward combinations.** In a K4 lifecycle question, an answer naming **one technique** is usually incomplete against one naming **a static review plus a dynamic test**.
- **Do not assume every architecture carries every risk.** The syllabus is specific about which risks it names for which architecture, and the official rationales are equally specific about the exclusions.
- **Chapter 3 is the worst points-per-hour chapter.** Budget accordingly: learn the architecture grid and the activity map thoroughly, and read the lifecycle sections once carefully rather than three times.

5.5 How to eliminate wrong answers

- Strike any option that puts **risk identification** anywhere but **test planning**, or **procedure ordering** anywhere but **implementation**.
- Strike any connectivity list that includes anything other than **mobile** and **embedded**.
- Strike any memory-leak list that includes **dynamic/cloud-based** or **mainframe**.
- Strike any language-comparison option that names something **the same in both languages** — latency, connectivity, batch processing.
- Strike any risk-timing option that **omits a lifecycle phase**.
- Strike any lifecycle-approach option that offers **only dynamic testing** when an option offering **static plus dynamic** exists.

- Strike any residual-risk statement without **numbers or thresholds**.
- Strike any option making the **vendor** responsible for validating **COTS** performance.

5.6 Scenario-solving shortcuts

- **Activity questions:** ask **what is being produced?** A plan and a risk list → **planning**. A decision about test types and conditions → **analysis**. A modular test case → **design**. An ordered procedure and a reset environment → **implementation**. A running test and a defect report → **execution**. A summary report and archived testware → **completion**. An action plan for a hardware change → **monitoring and control**.
- **Architecture questions:** ask **where does the code run and what does it depend on?** On one box → single computer. On several servers with fixed roles → multi-tier. On servers that change → distributed. On shared hardware → virtualized. On elastic infrastructure → cloud. Across a network to a client with its own code → client-server. On a battery → mobile. In a machine that could hurt someone → embedded. In a data center, decades old, now exposed via APIs → mainframe.
- **K4 risk-timing questions:** the correct option almost always contains the word **repeatedly** or spans **requirements through testing**.
- **K4 approach questions:** look for the option that does **something early and cheap** (a review) **and something late and thorough** (a full-data test).

5.7 Common candidate mistakes

- Studying the seven activities as definitions rather than as **task ownership**. The exam never asks what test planning is; it asks **which activity a named task belongs to**.
- Learning the nine architectures as descriptions without their **named risks**. Both official multiple-selection questions are pure risk-attribution.
- Forgetting the **non-architectural** risks — **C/C++ heap management**, **relational versus non-relational**, **recursive versus iterative** — even though one of the three PTFL-3.2 marks was spent there.
- Treating the K4 questions as harder versions of K2 questions. They are **scenario questions**, and their answers turn on **breadth** (all phases) and **combination** (static plus dynamic), not on obscure facts.
- Confusing **impact** and **likelihood** ownership. **Business stakeholders influence impact; technical stakeholders influence likelihood.**
- Over-investing in Chapter 3. At **2.2 marks per teaching hour**, it has the **lowest return in the syllabus** — learn it well, but not at Chapter 4's expense.
- Missing that **test monitoring and control** is where **exit criteria** are evaluated, not test completion.

5.8 Mnemonics that carry the whole chapter

Mnemonic	Unpacks to
P-M-A-D-I-E-C ("**P**lan, **M**onitor, **A**nalyse, **D**esign, **I**mplement, **E**xecute, **C**lose")	The seven principal activities in ISTQB test-process order. Ownership: **P** owns risk analysis and scope · **M** owns control measures and exit criteria · **A** owns the test basis, test conditions and test type · **D** owns modular test cases · **I** owns procedures, environment reset and production-like data · **E** owns the run and the defects · **C** owns the summary report, aggregation, dashboards and archiving.
"Plan the risk, implement the procedure"	The two official PTFL-3.1 answers in one phrase.
S-M-D-V-C-C-M-E-M	The nine architectures in syllabus order: **S**ingle computer · **M**ulti-tier · **D**istributed · **V**irtualized · **C**loud/dynamic · **C**lient-server · **M**obile · **E**mbedded · **M**ainframe.
"Mobiles and machines lose signal"	**Connectivity risk = mobile + embedded.** Nothing else.
"Leaks where memory is shared, small, or hand-managed"	**Memory-leak risk = virtualized (shared) + mobile (small) + client-server and embedded (C/C++, hand-managed).** Excluded: **cloud** (scales to compensate) and **mainframe** (managed memory).

Mnemonic	Unpacks to
"C counts, Python doesn't"	**C and C++ allow direct heap memory management**, so **memory use** must be monitored. Latency, connectivity and batch processing are language-independent.
I-A-M-M ("**I** Assess, **M**itigate, **M**anage")	The four risk steps: **I**dentify (time behavior, resource utilization, capacity) · **A**ssess (architecture categories; likelihood and impact; clearly defined criteria) · **M**itigate (nature of the item, level of risk) · **M**anage (ongoing, adequately mitigated **prior to release**).
"Business brings impact, technical brings likelihood"	The stakeholder split in **PTFL-3.3.1**, and the reason both groups are mandatory.
"Early and often, because risks move"	The K4 answer pattern: **started early and repeated regularly**, because **risks may change both in impact and likelihood**.
"Review first, then load"	The K4 approach pattern: **a technical review at design time** plus **a full-data performance test after implementation**.
"Verify four, conduct four, validate one"	The nine sequential phases: **verify** at concept, requirements, analysis and design, and coding · **conduct** at component, component integration, system and system integration · **validate** at acceptance.
"Cloud, subset, no-regression, slices"	The four CI concerns: **on-demand cloud environment · which tests to automate (a subset) · tests whose objective is that a change does not negatively impact performance · executing on portions of the system with selectable subsets**.

5.9 Chapter 3 mind map

CHAPTER 3 — Performance Testing in the Software Lifecycle · 195 minutes · 7 exam marks · K2 = 5, K4 = 2 · all objectives guaranteed

Branch	Node (K)	Everything hanging off it
3.1 Principal activities	**3.1.1** Principal performance testing activities (K2) — **2 marks**	**Iterative in nature** — each test's insights **correct or optimize parameters** until **test objectives are reached · aligns with the ISTQB test process · test planning** (**allocation of environments, data, tools, human resources**; **scope established**; **risk identification and analysis completed**; risks revisited) · **test monitoring and control** (**control measures / action plans** for **load generation capacity, hardware, network components, software implementation**; **exit criteria achievement**) · **test analysis** (**test basis**: requirements, objectives, **SLAs**, **IT architecture**, process models; **spreadsheets or capacity planning tools**; **load levels, timing conditions, transactions**; **the type of performance test is decided**) · **test design** (**modular** cases as **building blocks**) · **test implementation** (**ordered into procedures**; **environment reset before each execution**; **production-representative data in volume and type**) · **test execution** (conducted **often using performance test tools**; **defects reported**) · **test completion** (**test summary report** to **architects, managers, product owners**; **aggregated metrics**; **dashboards**; **test closure** archives **tests, tool assets, data and testware** for **maintenance**) · **ongoing at all test levels**
3.2 Architecture risks	**3.2.1** Categories of performance risks for different architectures (K2) — **3 marks**	**Performance varies by architecture, application and host environment**; **no complete list is possible · single computer** (**excessive resource consumption**: memory leaks, **background security software**, slow storage, **disk fragmentation**, **OS mismanagement**; **inefficient algorithms not using available resources**) · **multi-tier** (**poor or non-scalable database design**, **network bottlenecks**, **inadequate bandwidth or capacity on any single server**) · **distributed** (**servers change dynamically**; **unreliable or unpredictable remote servers**, **periodic connection problems**, **intermittent intense load**) · **virtualized** (**excessive load across all VMs**; **improper host VM configuration**) · **dynamic/cloud** (**self-scaling mitigates**, but **misconfiguration at setup or**

Branch	Node (K)	Everything hanging off it
		update) · **client-server** (client code carries single-computer risks; **connection speed and reliability**, **public Wi-Fi congestion**, **firewalls, packet inspection, load balancing**) · **mobile** (**location, battery life, charge state, memory, temperature**; **accelerometers, Bluetooth**; **heavy interaction with local apps and remote web services**) · **embedded real-time** (**cars, elevators, traffic signals, HVAC**; **connectivity via IoT**; **slowdowns can be catastrophic**) · **mainframe** (**predictable as designed**, but **APIs, web services and database access bring unexpected loads**) · **architectures combine — all relevant risks apply · non-architectural risks**: **C/C++ heap management**, **relational vs non-relational**, **recursive vs iterative**
3.3 Risks across the SDLC	**3.3.1** Analyze performance risks across the lifecycle (**K4**) — **1 mark**	**Four steps**: **identify** (time behavior, resource utilization, capacity) → **assess** (**relevant architecture categories**; **likelihood and impact**; **clearly defined criteria**) → **mitigate** (**nature of the risk item and the level of risk**) → **manage** (**ongoing**, **adequately mitigated prior to release**) · **both business and technical stakeholders**: business must appreciate **intended usage, business/societal/safety criticality, financial and reputational damage, civil or criminal legal liability** → **impact**; technical must appreciate **requirements, architecture, design and implementation decisions** → **likelihood · appropriate formality and rigor · started early, repeated regularly · do not rely on late system and system integration testing · many-to-many joins** example: **only revealed in dynamic testing with large datasets**, but **predictable by technical review with experienced database engineers · mitigation spans the entire development process** — e.g. **design for highly variable scalability (on-demand cloud) when volumes cannot be specified early · residual risk always exists** and must be **realistic and specific** — **percentage of customers exceeding thresholds**, not "it's still possible"
3.4 Lifecycle activities	**3.4.1** Determine appropriate activities per lifecycle phase (**K4**) — **1 mark**	**Sequential** — ideal practice: **performance criteria as acceptance criteria at the outset**, activities **throughout the lifecycle**, each **based on prior activities** · nine phases: **concept** (goals as acceptance criteria) · **requirements** (defined and representing stakeholder needs) · **analysis and design** (design reflects requirements) · **coding** (code efficient, reflects requirements and design) · **component · component integration · system** (**hardware, software, procedures and data representative of production**; **interfaces may be simulated if they give a true representation of performance**) · **system integration** (**entire system, representative of production**) · **acceptance** (**validate against originally stated user needs and acceptance criteria**) · **iterative/incremental** — first iteration or a **dedicated performance iteration**; **execution may be by a separate team · CI**: objective is **regression and build stability**; four extra concerns — **on-demand cloud environment**, **which tests to automate (a subset, short timeframe, specialist team elsewhere)**, **tests to ensure a change does not negatively impact performance**, **execution on portions with selectable subsets · seven iterative activities**: **release planning · iteration planning · user story creation (non-functional stories) · design of performance tests · coding (algorithm tuning) · testing/evaluation (possibly a dedicated iteration) · delivery (monitor in production) · COTS / supplier-acquirer**: testing needed **from both perspectives**; **the customer usually validates**; **contract** for customized vendor software; **sole customer responsibility** for COTS, **in a realistic test environment prior to deployment**

Section 6 — Flashcards

Cover the BACK row and recall it from the FRONT. All 45 cards are drawn from Chapter 3 of the CT-PT 2018 syllabus. Target: three clean passes before exam day.

FRONT	**1. Describe the iterative nature of performance testing.**
BACK	**Each test provides valuable insights**; information from one test is used to **correct or optimize application and system parameters**; the next iteration **shows the results of modifications**, and so on **until test objectives are reached**.
FRONT	**2. Which process do performance testing activities align with?**
BACK	**The ISTQB test process.**
FRONT	**3. Why is test planning particularly important for performance testing?**
BACK	Because of the need for the **allocation of test environments, test data, tools and human resources**. It is also where **the scope of performance testing is established**.
FRONT	**4. Which activity completes risk identification and risk analysis?**
BACK	**Test planning.** Relevant information is updated in test planning documentation (e.g., **test plan, level test plan**), and **risks, risk levels and risk status are modified** as conditions change.
FRONT	**5. Name the four control measures listed under test monitoring and control.**
BACK	**Increasing the load generation capacity** if the infrastructure does not generate the desired loads · **changed, new or replaced hardware · changes to network components · changes to software implementation**.
FRONT	**6. Where are the performance test objectives evaluated for exit criteria achievement?**
BACK	**Test monitoring and control.**
FRONT	**7. What does test analysis analyse, and what does it decide?**
BACK	It analyses **performance requirements, test objectives, SLAs, IT architecture, process models and other test basis items** — possibly using **spreadsheets or capacity planning tools** — and identifies **load levels, timing conditions and transactions to be tested**. **The required type(s) of performance test are then decided.**
FRONT	**8. How are performance test cases generally created during test design?**
BACK	**In modular form**, so they can be used as the **building blocks of larger, more complex performance tests**.
FRONT	**9. What happens during test implementation?**
BACK	**Performance test cases are ordered into performance test procedures** reflecting **the steps normally taken by the user**; the **test environment is established and/or reset before each execution**; and **test data representative of actual production data in volume and type** is established.
FRONT	**10. What is produced at test completion, and for whom?**
BACK	A **test summary report** for stakeholders such as **architects, managers and product owners**, with results **expressed through metrics which are often aggregated**, and often presented via **dashboards**.
FRONT	**11. What is the point of test closure?**
BACK	The stage where **designed tests, test tool assets (test cases and test procedures), test data and other testware are archived or passed on to other testers for later use during system maintenance activities**.
FRONT	**12. At which test levels is performance testing performed?**
BACK	It is an **ongoing activity performed at multiple times and at all test levels — component, integration, system, system integration and acceptance testing**.
FRONT	**13. Define a single computer system and name its two risk categories.**
BACK	A system or application that runs **entirely on one non-virtualized computer**. Risks: **excessive resource consumption**

	(memory leaks, **background activities such as security software**, slow storage subsystems, **disk fragmentation**, **operating system mismanagement**) and **inefficient implementation of algorithms** that **do not make use of available resources** and so **execute slower than required**.
FRONT	**14. Name the multi-tier system risks.**
BACK	**Poor or non-scalable database design · network bottlenecks · inadequate bandwidth or capacity on any single server** — plus **all the single-computer risks per server**.
FRONT	**15. What distinguishes a distributed system, and what extra risk does it carry?**
BACK	**The various servers may change dynamically** (e.g., **different inventory databases depending on geographic location**). Extra risk: **critical workflows or dataflows to, from, or through unreliable or unpredictable remote servers**, especially with **periodic connection problems or intermittent periods of intense load**.
FRONT	**16. Name the two virtualization-specific risks.**
BACK	**Excessive load on the hardware across all the virtual machines**, and **improper configuration of the host virtual machine resulting in inadequate resources**.
FRONT	**17. What risk do dynamic / cloud-based systems carry despite their self-scaling features?**
BACK	**Failures to properly configure these features during initial setup or subsequent updates.**
FRONT	**18. Name the client-server risks beyond the single-computer risks of client-side code.**
BACK	**Connection speed and reliability issues · network congestion at the client connection point (e.g., public Wi-Fi) · potential problems due to firewalls, packet inspection and server load balancing.**
FRONT	**19. Which five factors affect the limited and variable resources and connectivity of a mobile device?**
BACK	**Location · battery life · charge state · available memory on the device · temperature.**
FRONT	**20. What extra mobile risks arise from sensors and integrations?**
BACK	**Slow dataflows** from **device sensors or radios such as accelerometers or Bluetooth**, and **heavy interactions with other local mobile apps and remote web services**, any of which can become a **performance efficiency bottleneck**.
FRONT	**21. How do embedded real-time systems differ from mobile in risk consequence?**
BACK	They share many risks including **connectivity**, but **the diminished performance of a mobile video game is usually not a safety hazard, while such slowdowns in a vehicle braking system could prove catastrophic**.
FRONT	**22. Give three examples of embedded real-time systems from the syllabus.**
BACK	Any three of: **cars (entertainment systems and intelligent braking systems) · elevators · traffic signals · HVAC systems**.
FRONT	**23. What is the modern performance risk for mainframe applications?**
BACK	**Many are now accessible via APIs, web services, or through their database, which can result in unexpected loads that affect throughput of established applications.**
FRONT	**24. Which environments are most likely to have connectivity risk?**
BACK	**Mobile and embedded — particularly when embedded software is used in IoT devices**.
FRONT	**25. Which environments are most likely to have memory-leak risk, and which two are excluded?**
BACK	**Virtualized · client/server and browser-based · mobile · embedded.** Excluded: **dynamic/cloud-based** (**the environment will scale to compensate**) and **mainframe** (**older code in higher level languages that provide their own memory management**).
FRONT	**26. What must you monitor for C++ that you would not for Python, and why?**
BACK	**Memory use — C and C++ require the programmer to do their own memory management, which sometimes results in memory leaks (where memory is allocated, but not de-allocated)**.
FRONT	**27. Name the three non-architectural technical decisions that create performance risk.**

BACK	**Languages allowing direct heap memory management (C, C++) · relational versus non-relational databases · the choice of a recursive as opposed to an iterative algorithm**.
FRONT	**28. What happens when an application incorporates two or more architectures?**
BACK	**All relevant risks will apply.** With **IoT and mobile**, **it is possible that all architectures are present in some form and thus all risks can apply**.
FRONT	**29. List the four steps of the performance risk process.**
BACK	**1 Identify** risks to product quality (time behavior, resource utilization, capacity) · **2 Assess**, addressing the relevant architecture categories and evaluating **likelihood and impact using clearly defined criteria · 3 Mitigate** based on the **nature of the risk item and the level of risk · 4 Manage on an ongoing basis** to ensure risks are **adequately mitigated prior to release**.
FRONT	**30. What must business stakeholders appreciate?**
BACK	That **intended usage, business-, societal-, or safety-criticality, potential financial and/or reputational damage, civil or criminal legal liability** and similar factors **create risks and influence the impact of failures**.
FRONT	**31. What must technical stakeholders appreciate?**
BACK	That **architecture, design, and implementation decisions affect performance risks from a technical perspective, creating risks and influencing the likelihood of defects**.
FRONT	**32. What is the syllabus's guidance on the timing of performance risk analysis?**
BACK	It must be **started early and repeated regularly**; **the tester should avoid relying entirely on performance testing conducted towards the end of the system test level and system integration test level**.
FRONT	**33. Give the many-to-many join example.**
BACK	**Slow performance of many-to-many joins due to poor database design may only reveal itself during dynamic testing with large-scale test datasets, such as those used during system test. However, a careful technical review that includes experienced database engineers can predict the problems prior to database implementation.**
FRONT	**34. What should be done when transaction volumes or simultaneous-user counts cannot be specified early?**
BACK	**Design and architecture decisions should allow for highly variable scalability (e.g., on-demand cloud-based computing resources)**, which **enables early risk mitigation decisions to be made**.
FRONT	**35. How should residual risk be communicated?**
BACK	As a **realistic and specific evaluation** — e.g., **the percentage of customers likely to experience delays equal to or exceeding certain thresholds**, not "**yes, it's still possible for customers to experience long delays**".
FRONT	**36. What is ideal practice for performance testing in sequential models?**
BACK	**Include performance criteria as part of the acceptance criteria which are defined at the outset of a project**, and **conduct performance testing activities throughout the software development lifecycle**, each activity **based on items defined in the prior activities**.
FRONT	**37. What are the performance activities at the concept and requirements phases?**
BACK	**Concept: verify that system performance goals are defined as acceptance criteria for the project. Requirements: verify that performance requirements are defined and represent stakeholder needs correctly.**
FRONT	**38. What is permitted at system testing that is not permitted at system integration testing?**
BACK	At **system testing**, **system interfaces may be simulated provided that they give a true representation of performance**. At **system integration testing**, performance testing is conducted **with the entire system which is representative of the production environment**.
FRONT	**39. What is the activity at acceptance testing, and which verb does the syllabus use?**
BACK	**Validate** that **system performance meets the originally stated user needs and acceptance criteria**.

FRONT	**40. How can performance testing be organised in iterative and incremental models?**
BACK	**As part of the first iteration, or as an iteration dedicated entirely to performance testing** — and **execution may be performed by a separate team tasked with performance testing**.
FRONT	**41. What is the most common objective of testing in CI, and what is the objective of CI performance tests?**
BACK	CI's most common objective is **regression testing and ensuring each build is stable**. The main objective of **CI performance tests** is **to ensure a change does not negatively impact performance**.
FRONT	**42. List the four additional CI concerns for performance testing.**
BACK	**Setup of the performance test environment** (often needs an **on-demand, cloud-based** environment) · **determining which performance tests to automate** (a **subset**, given the **short timeframe**) · **creating the tests** (**new tests may be required** depending on the build's changes) · **executing on portions of the application or system** (needs **rapid testing** and **selectable subsets**).
FRONT	**43. Name the seven iterative lifecycle activities.**
BACK	**Release planning · iteration planning · user story creation · design of performance tests · coding/implementation · testing/evaluation · delivery.**
FRONT	**44. What are "non-functional" user stories?**
BACK	User stories that **form the basis of performance requirements in Agile methodologies**, with the **specific performance criteria described in the associated acceptance criteria**.
FRONT	**45. Who is responsible for performance in COTS and supplier/acquirer models?**
BACK	**Regardless of the source, it is often the responsibility of the customer to validate that the performance meets their requirements.** For **customized vendor-developed software**, criteria **should be specified as part of the contract**. For **COTS**, **the customer has sole responsibility to test the performance of the product in a realistic test environment prior to deployment**.

Section 7 — Revision Notes

7.1 One-page chapter summary

Chapter 3 — Performance Testing in the Software Lifecycle

Performance testing is iterative: each test's insights **correct or optimize application and system parameters** until **test objectives are reached**, and its activities **align with the ISTQB test process**. **Test planning** — particularly important because of the **allocation of environments, data, tools and human resources** — is where **the scope is established** and where **risk identification and analysis are completed**. **Test monitoring and control** defines **action plans** for load-generation shortfalls, hardware, network and software changes, and **evaluates exit criteria**. **Test analysis** works from the **test basis** — requirements, objectives, **SLAs**, **IT architecture**, process models — to identify **load levels, timing conditions and transactions**, and **decides the type of performance test**. **Test design** produces **modular** cases as **building blocks**; **test implementation** orders them into **procedures**, **resets the environment** and establishes **production-representative data**; **test execution** runs and reports; **test completion** issues the **test summary report** with **aggregated metrics** and **dashboards**, and **archives testware at test closure**. Nine **architectures** each carry named risks: **single computer** (excessive resource consumption, inefficient algorithms) · **multi-tier** (database design, network bottlenecks, server capacity) · **distributed** (**unreliable remote servers**, dynamic server membership) · **virtualized** (**load across all VMs**, host misconfiguration) · **dynamic/cloud** (**self-scaling misconfiguration**) · **client-server** (**connection speed, public Wi-Fi, firewalls, packet inspection, load balancing**) · **mobile** (**location, battery, charge state, memory, temperature**, sensors, app and web-service interactions) · **embedded real-time** (mobile risks, but **catastrophic** rather than annoying) · **mainframe** (**newly exposed via APIs, web services and the database**). **Connectivity risk means mobile and embedded**; **memory-leak risk means virtualized, client-server, mobile and embedded** — cloud **scales to compensate** and mainframes use **managed-memory languages**. Risk is not only architectural: **C and C++ allow direct heap memory management**, **relational and non-relational databases behave differently**, and even **recursive versus iterative algorithms** matter. Across the lifecycle, the process is **identify → assess → mitigate → manage**, with **business stakeholders influencing impact** and **technical stakeholders influencing likelihood**; it must be **started early and repeated regularly**, because **risks change in both impact and likelihood** — the **many-to-many join** example shows a defect that surfaces only at **system test** yet is **predictable by technical review**. **Residual risk always remains** and must be given **thresholds and percentages**, not reassurance. Finally, lifecycle models shape the work: **sequential** has **nine phases — verify four, conduct four, validate one**, with **simulation permitted only at system testing**; **iterative** adds **seven activities** and **four CI concerns**, with performance criteria carried in **non-functional user stories**; and **COTS** places **sole responsibility on the customer** to test **in a realistic test environment prior to deployment**.

7.2 Key definitions to memorize

- **Test planning** — establishes the **scope**, allocates **environments, data, tools and human resources**, and completes **risk identification and analysis**.
- **Test monitoring and control** — defines **control measures / action plans** and evaluates **exit criteria achievement**.
- **Test analysis** — analyses the **test basis**, identifies **test conditions** (load levels, timing conditions, transactions) and **decides the performance test type**.
- **Test implementation** — orders test cases into **procedures**, **resets the environment**, establishes **production-representative test data**.
- **Test completion** — **test summary report**, **aggregated metrics**, **dashboards**, **testware archived at the point of test closure**.
- **Connectivity-risk architectures** — **mobile** and **embedded**.
- **Memory-leak-risk architectures** — **virtualized**, **client-server and browser-based**, **mobile**, **embedded**.
- **The four-step risk process** — **identify · assess · mitigate · manage**.
- **Residual risk** — always exists; must be a **realistic and specific evaluation**, ideally **percentages against thresholds**.
- **Non-functional user story** — an Agile story whose **performance criteria live in its acceptance criteria**.

7.3 High-priority concepts (study first)

Priority	Concept	LO	Marks
★★★	The nine architectures with their **named** risks, plus the **connectivity** and **memory-leak** groupings	PTFL-3.2.1	**3**
★★★	Which of the seven activities owns which task — especially **planning owns risk, implementation owns procedures**	PTFL-3.1.1	**2**
★★★	The K4 timing answer: **repeatedly throughout requirements, development and performance testing**	PTFL-3.3.1	1
★★★	The K4 approach answer: **technical review plus full-data performance test**	PTFL-3.4.1	1
★★	The **C/C++ versus Python** memory-management distinction	PTFL-3.2.1	—
★★	**Business = impact · technical = likelihood**	PTFL-3.3.1	—
★★	The nine sequential phases — **verify, conduct, validate** — and where **simulation** is allowed	PTFL-3.4.1	—
★	The four CI concerns and the seven iterative activities	PTFL-3.4.1	—
★	COTS: **sole customer responsibility, realistic environment, prior to deployment**	PTFL-3.4.1	—

7.4 Last-minute revision sheet

- **7 of 40 questions · 7 of 40 points · K2 = 5, K4 = 2 · all guaranteed · 195 minutes of teaching.** Sample Exam questions **#11–#17**. **PTFL-3.2 is worth 3; PTFL-3.1 is worth 2.**
- **Activity ownership:** planning = **risk + scope + resources** · monitoring and control = **action plans + exit criteria** · analysis = **test basis + conditions + test type** · design = **modular cases** · implementation = **procedures + environment reset + production-like data** · execution = **run + defects** · completion = **summary report + aggregation + dashboards + archiving**.
- **Connectivity → mobile, embedded. Memory leaks → virtualized, client-server, mobile, embedded. Not cloud** (scales to compensate), **not mainframe** (managed-memory languages).
- **C/C++ → monitor memory use.** Latency, connectivity and batch processing are the same in any language.
- **Risk process:** identify (**time behavior, resource utilization, capacity**) → assess (**architecture categories; likelihood and impact; clearly defined criteria**) → mitigate (**nature and level**) → manage (**ongoing, prior to release**).
- **Business stakeholders → impact. Technical stakeholders → likelihood.**
- **Start early, repeat regularly.** Risks **change in both impact and likelihood**. Do not rely on **late system and system integration testing**.
- **Residual risk:** give **percentages against thresholds**, not "it's still possible".
- **Sequential:** verify at **concept, requirements, analysis and design, coding**; conduct at **component, component integration, system, system integration**; **validate** at **acceptance**. **Simulated interfaces are allowed only at system testing, and only if they give a true representation of performance.**
- **CI concerns: on-demand cloud environment · a subset of tests · tests that prove a change does not degrade performance · execution on portions with selectable subsets.**
- **COTS: the customer has sole responsibility**, **in a realistic test environment**, **prior to deployment**.

7.5 Memorization checklist

1. I can name the four learning objectives and say which two are **K4** and which one is worth **three marks**.
2. I can name the seven principal activities and the tasks each one owns.
3. I can say which activity completes **risk identification and analysis** and which one **orders test cases into procedures**.
4. I can list the nine architectures in syllabus order.

5. I can give at least two named risks for each architecture.
6. I can state which architectures carry **connectivity** risk and which carry **memory-leak** risk, with the reason for each exclusion.
7. I can explain what must be monitored for **C++** that need not be for **Python**, and why.
8. I can name the three **non-architectural** technical decisions that create performance risk.
9. I can recite the **four-step** risk process with the content of each step.
10. I can state which stakeholder group influences **impact** and which influences **likelihood**.
11. I can quote the guidance on **timing** — started early, repeated regularly — and explain **why risks are re-assessed**.
12. I can give the **many-to-many join** example and explain what it proves about static testing.
13. I can state how **residual risk** should and should not be communicated.
14. I can list the **nine sequential phases** with the correct verb for each.
15. I can say where **simulated interfaces** are permitted and under what condition.
16. I can list the **four CI concerns** and the **seven iterative activities**.
17. I can state the responsibility split for **customized vendor software** and for **COTS**.

7.6 Exam-day quick review

60-second mental checklist before answering any Chapter 3 question

1) Which LO is this? (activities · architecture risks · risk across the lifecycle · lifecycle activities) **2)** If it names a **task** and asks **when**, map it to one of the seven activities — **risk to planning, procedures to implementation, exit criteria to monitoring and control**. **3)** If it lists **numbered environments**, work by **exclusion**: strike cloud and mainframe for memory leaks; strike everything but mobile and embedded for connectivity. **4)** If it compares **languages**, ask whether the concern is language-dependent — only **memory management** is. **5)** If it asks **when to analyse risk**, choose the option that spans **the whole lifecycle** and says **repeatedly**. **6)** If it asks the **best way** to address a performance concern, choose the option pairing an **early static review** with a **later dynamic test**. **7)** Check qualifiers — **representative of production**, **a true representation of performance**, **prior to deployment**, **prior to release**. **8)** If two options survive, choose the one that repeats **syllabus wording**.

Sources: ISTQB® Certified Tester Foundation Level Specialist Syllabus — Performance Testing, Version 2018 (9 December 2018), Chapter 3 and Keywords; ISTQB® CT-PT Sample Exam Questions and Sample Exam Answers, Version 2018 (9 December 2018); ISTQB® Exam Structure Tables v1.16 (CT-PT) and ISTQB® Exam Structures and Rules v1.2. All content is traceable to these references; no external knowledge was used.